

Animation and Kinematics



Some Slides/Images adapted from Marschner and Shirley and David Levin

Animation and Kinematics

Agenda:

- Animation in Computer Graphics
- Forward Kinematics
- Skinning for Mesh Deformation
- Keyframe Animation + Splines
- Inverse Kinematics

Animation Timeline

1908: Emile Cohl (1857-1938) France, makes his first film, FANTASMAGORIE, arguably the first animated film.

1911: Winsor McCay (1867-1934) makes his first film, LITTLE NEMO. McCay, already famous for comic strips, used the film in his vaudeville act. His advice on animation:

Any idiot that wants to make a couple of thousand drawings for a hundred feet of film is welcome to join the club.

1928: Walter Disney (1901-1966) working at the Kansas City Slide Company creates Mickey Mouse.

1974: First Computer animated film "Faim" from NFB nominated for an Oscar.

Animation Principles

Squash & Stretch

Timing

Ease-In & Ease-Out

Arcs

Anticipation

Follow-through & Secondary
Motion

Overlapping Action & Asymmetry

Exaggeration

Staging

Appeal

Straight-Ahead vs. Pose-to-Pose

Case Study: Squash and Stretch

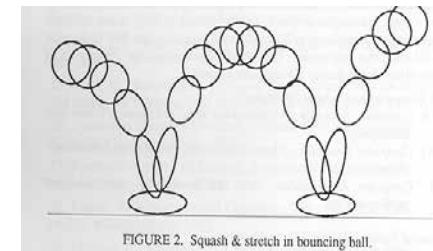
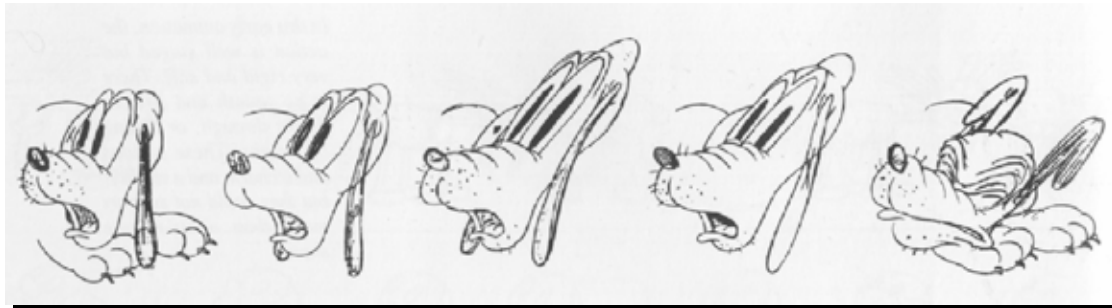
Rigid objects look robotic: deformations make motion natural

Accounts for physics of deformation

- Think squishy ball...
- Communicates to viewer what the object is made of, how heavy it is, ...
- Usually large deformations conserve volume: if you squash one dimension, stretch in another to keep mass constant

Also accounts for persistence of vision

- Fast moving objects leave an elongated streak on our retinas



What can be animated?

- Lights
- Camera
- **Jointed figures**
- **Skin/muscles**
- **Deformable objects**
- Clothing
- Wind/water/fire/smoke
- Hair

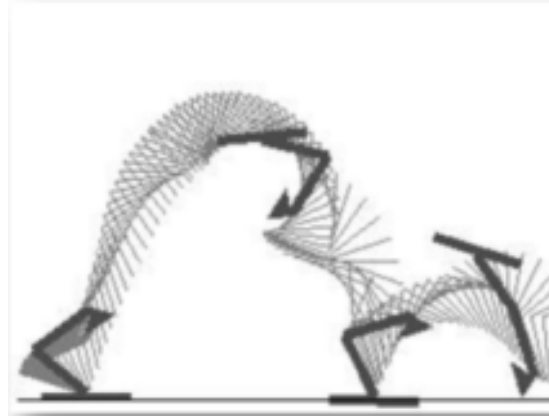
...any variable, Given the right time scale, almost anything...

Approaches to Animation

How does one make digital models move?



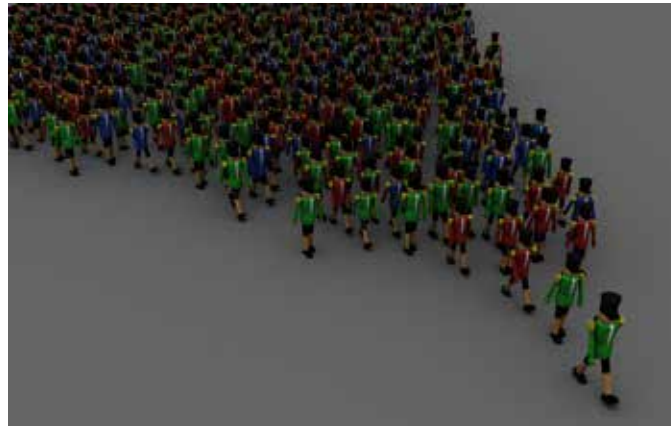
Keyframing



Physical simulation



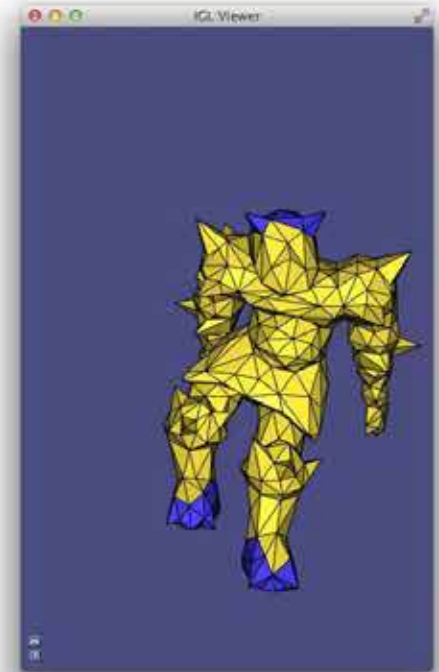
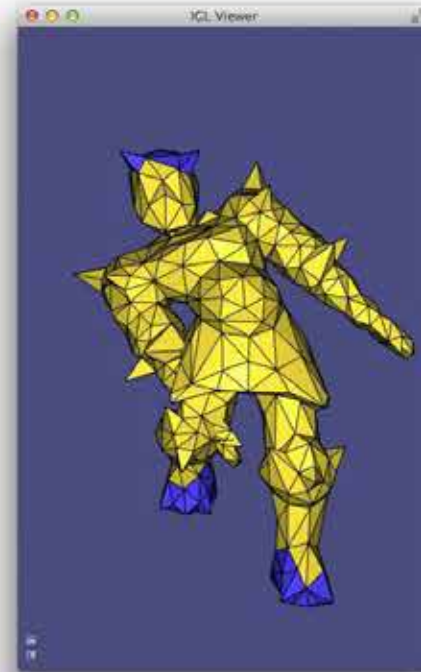
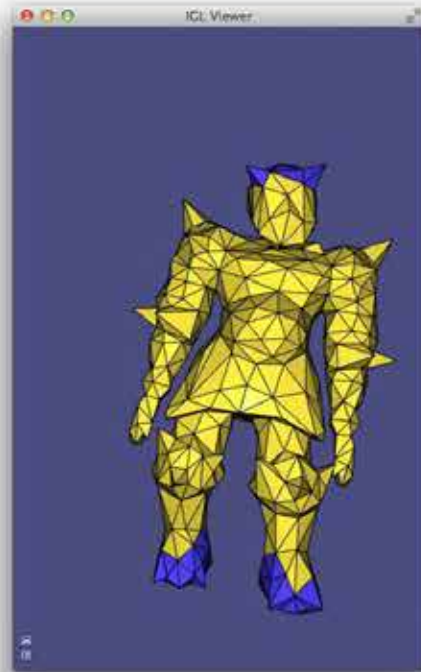
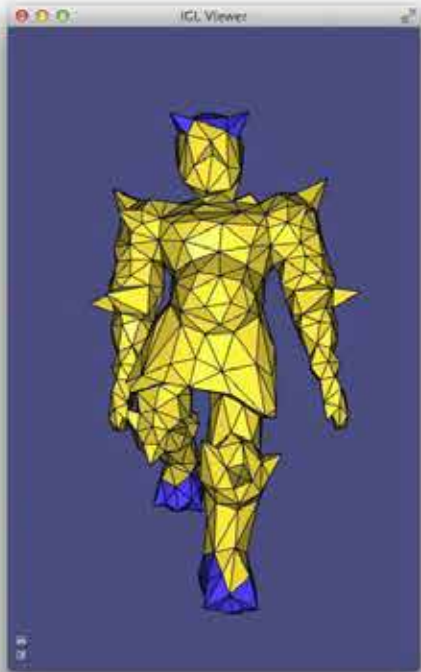
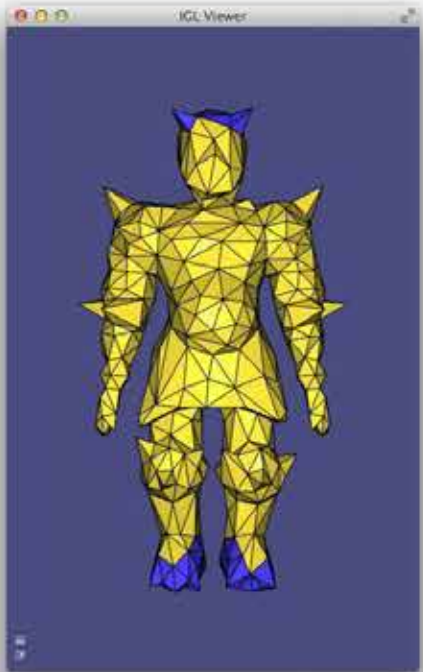
Motion capture



Behavior rules

How do articulated characters move?

What to keyframe/simulate/animate?



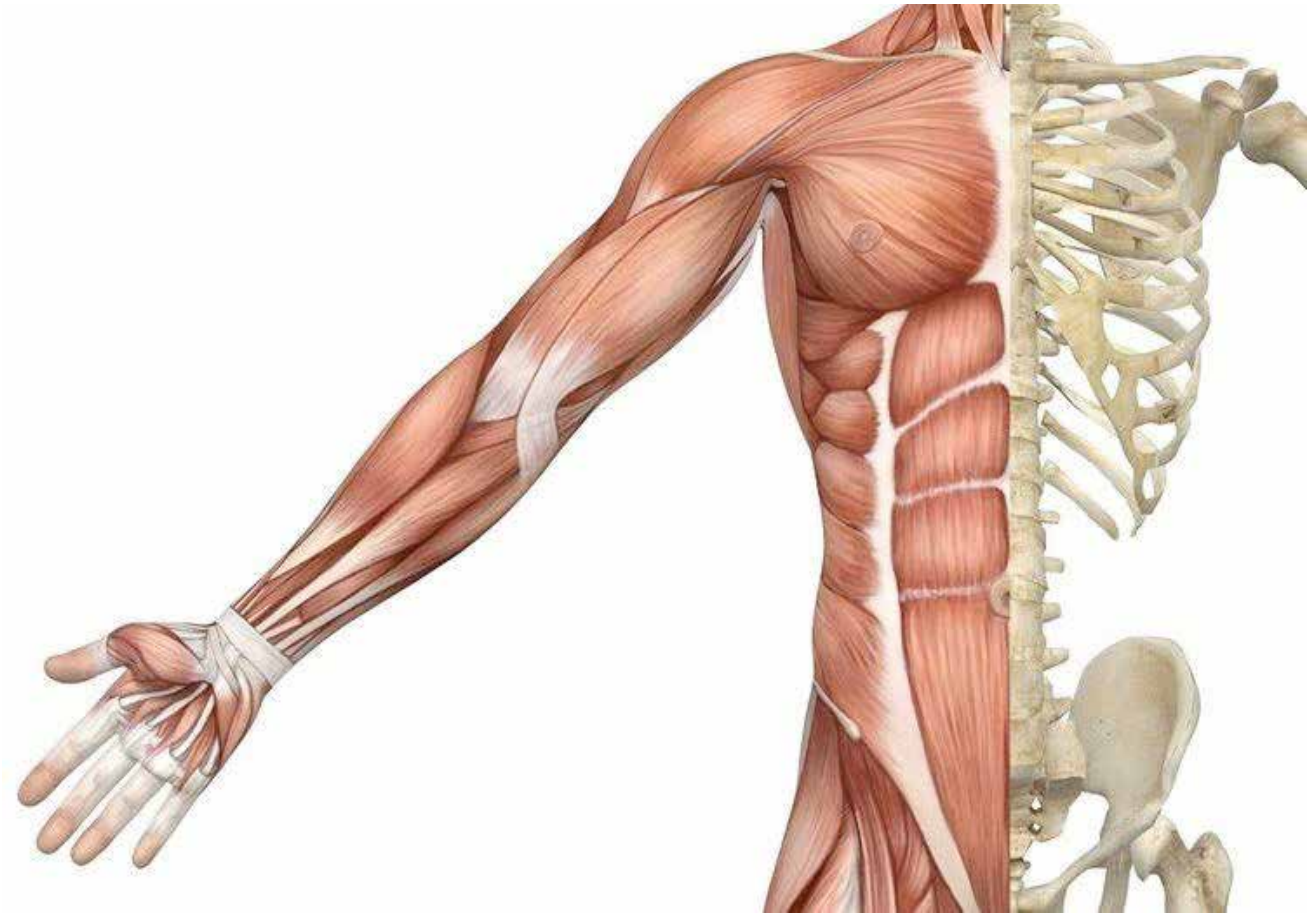
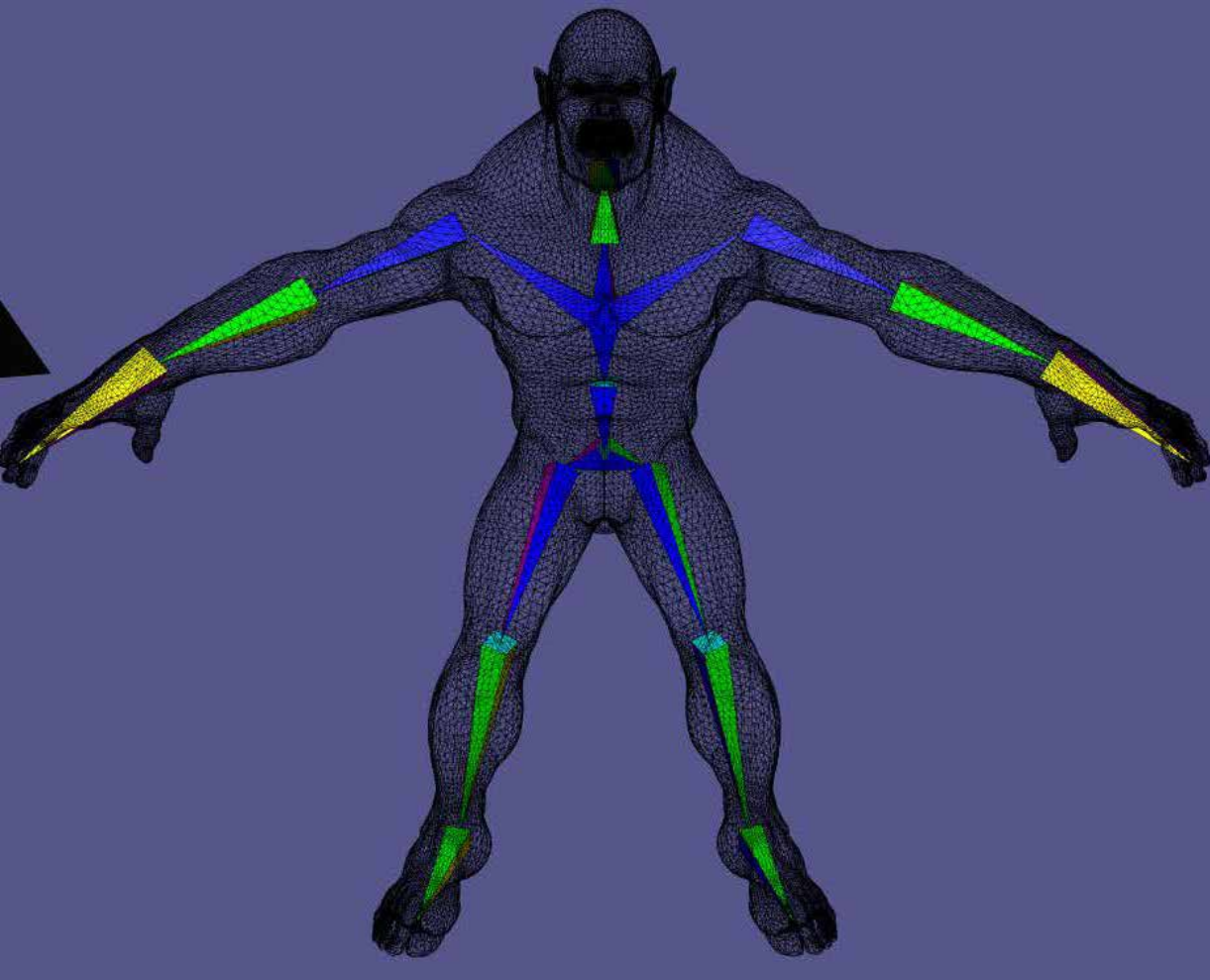
Per-Vertex?

Animation and Kinematics

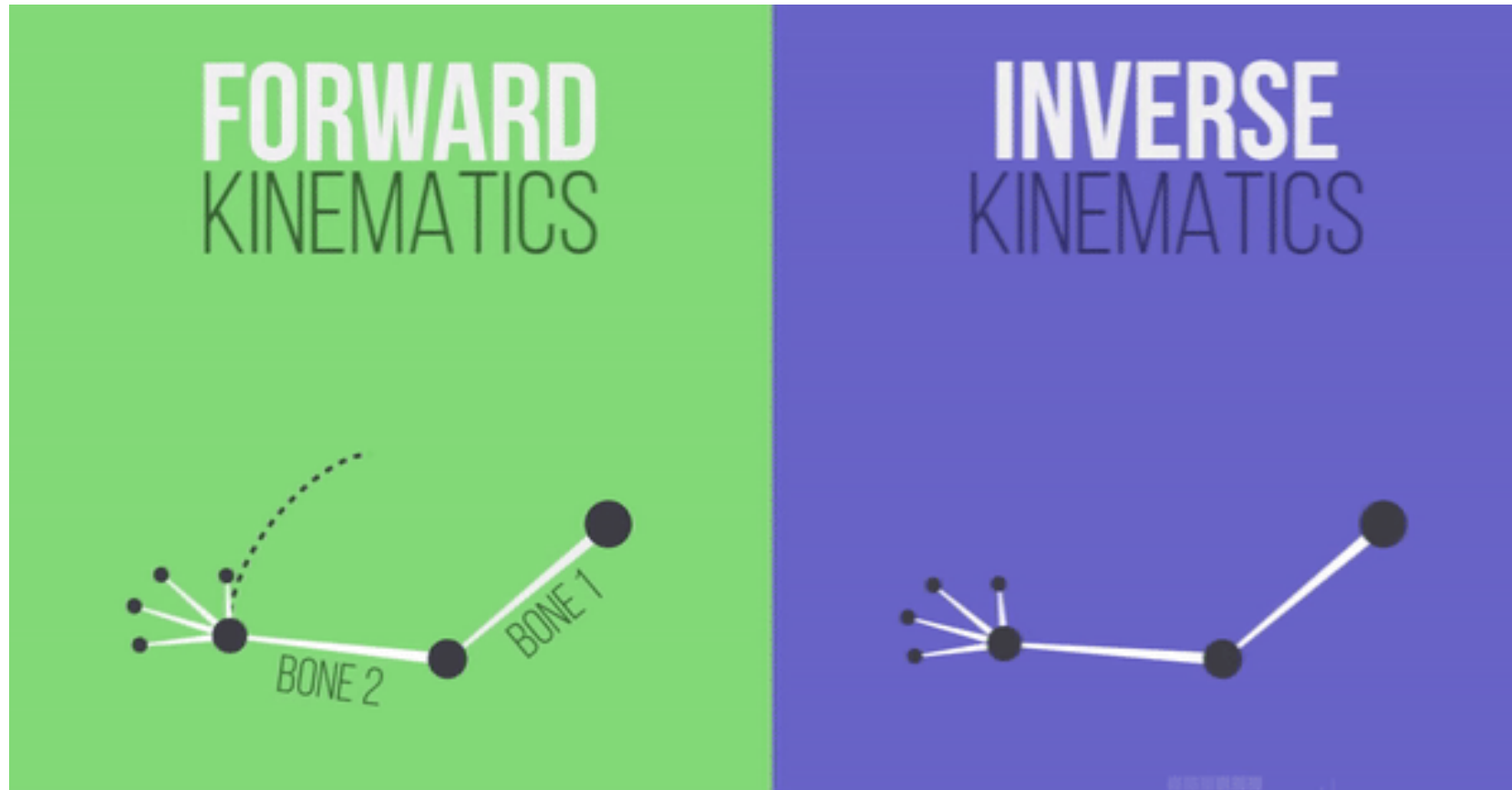
Agenda:

- Animation in Computer Graphics
- Forward Kinematics
- Skinning for Mesh Deformation
- Keyframe Animation + Splines
- Inverse Kinematics

Skeleton: hierarchy of bones/joints



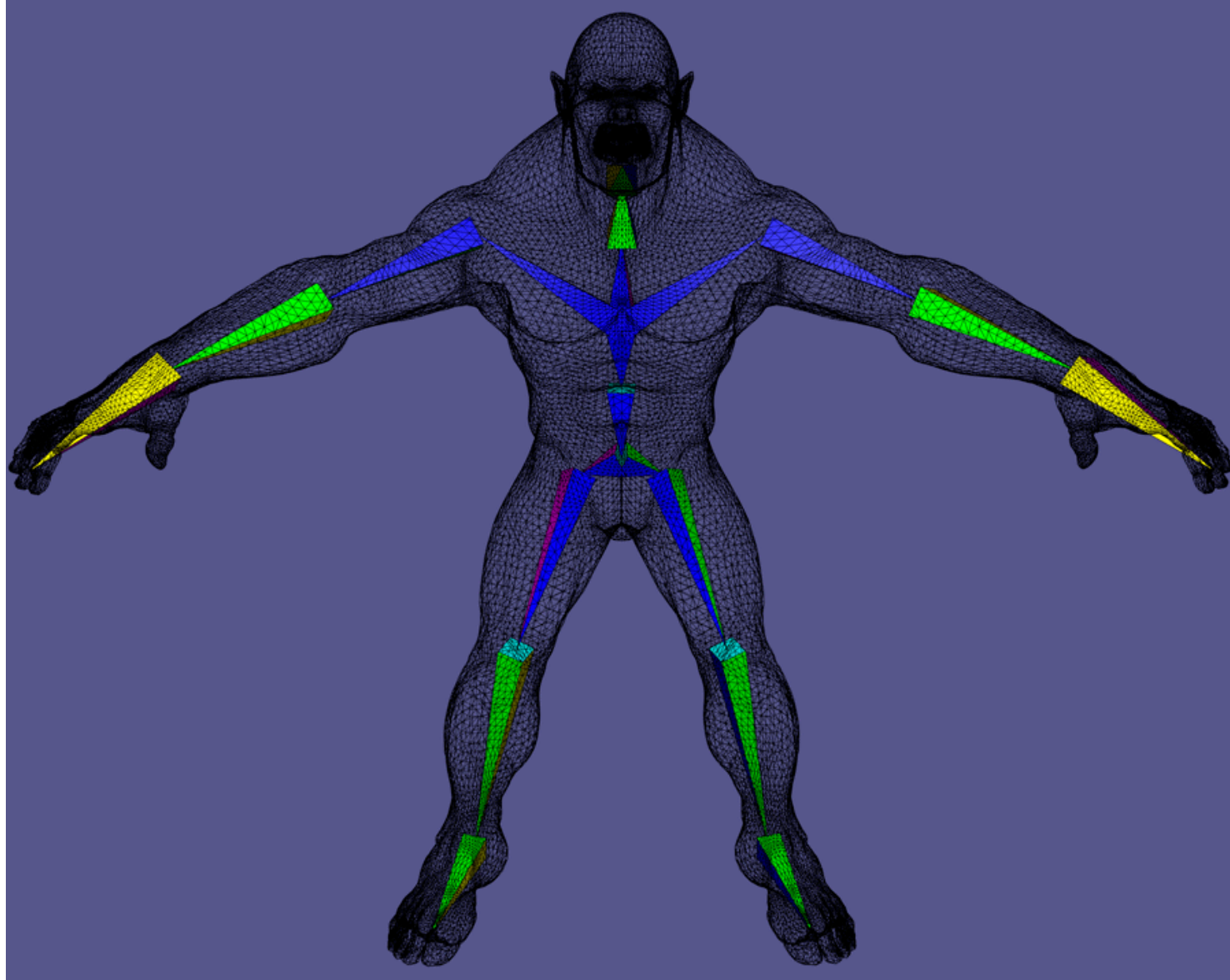
Posing a skeleton: Forward Kinematics vs. Inverse Kinematics



Skeletons



Deforming the object: Skinning



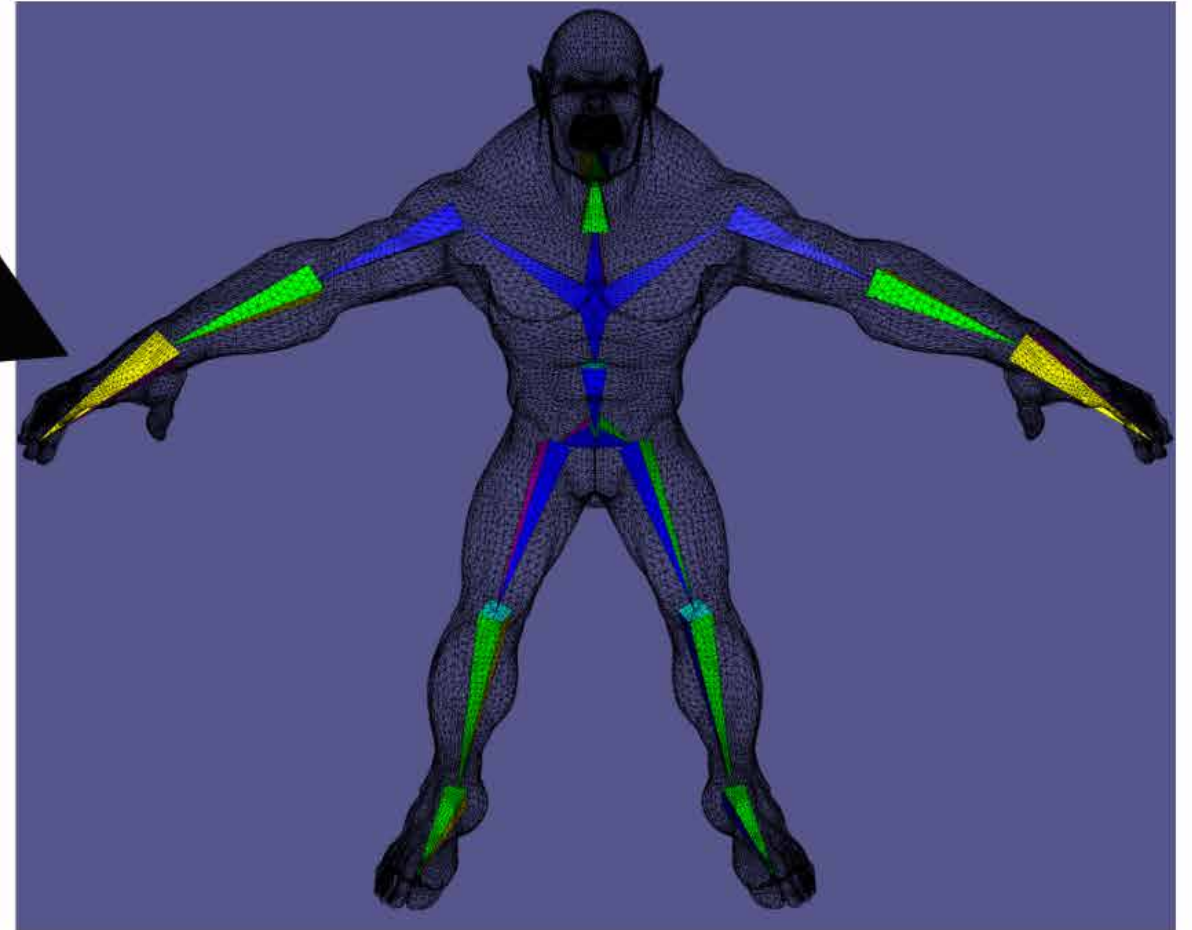
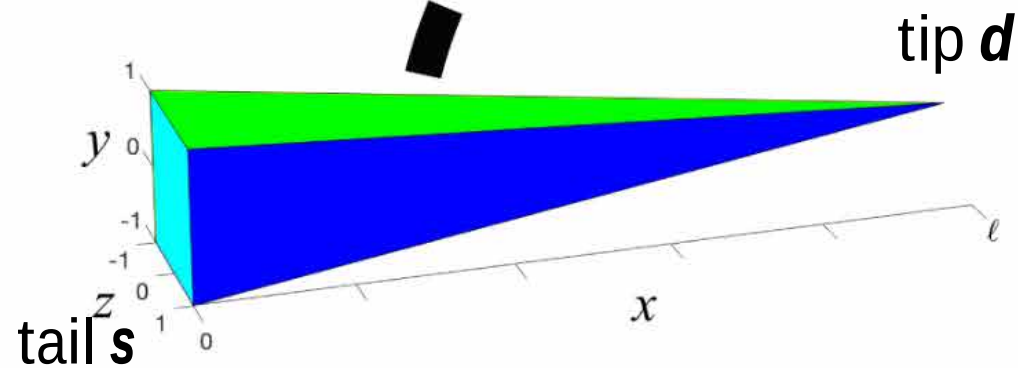
Skeletons: Rest Bone

$$t=s=d_p$$

$$d=R[l \ 0 \ 0]^T + d_p$$

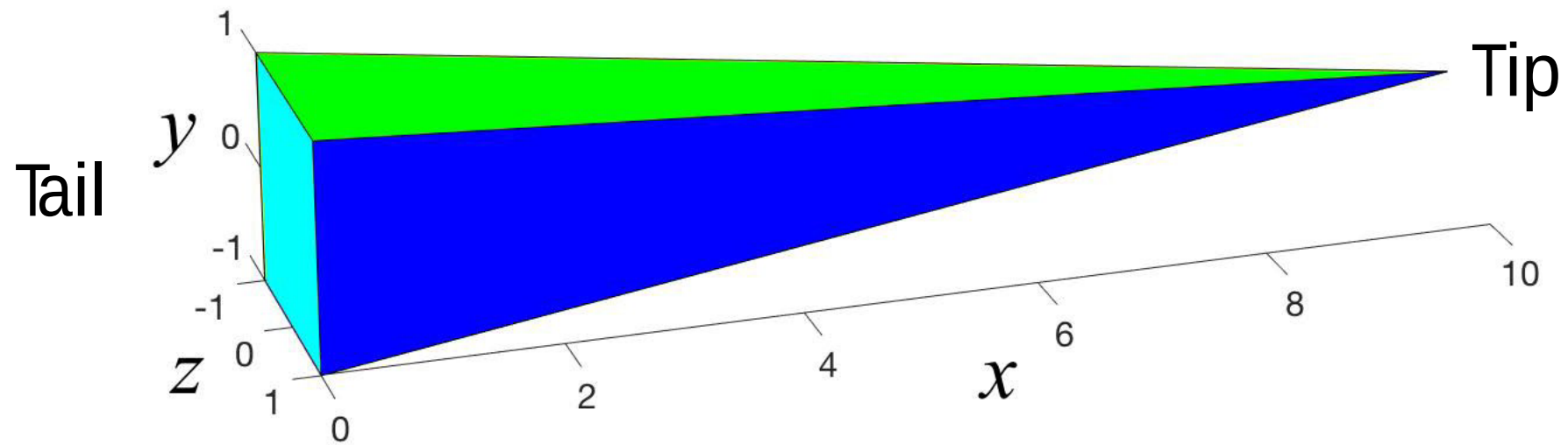
$$\hat{\mathbf{T}} = (\hat{\mathbf{R}} \quad \hat{\mathbf{t}}) \in \mathbb{R}^{3 \times 4}$$

Bone of length ℓ :

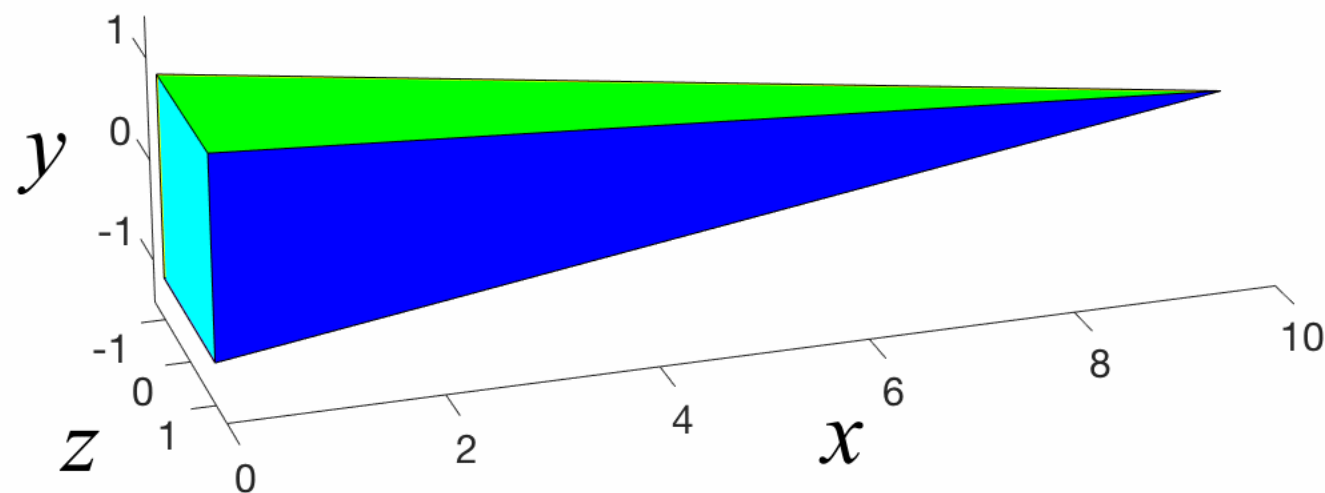


Skeletons: Specifying Rotations

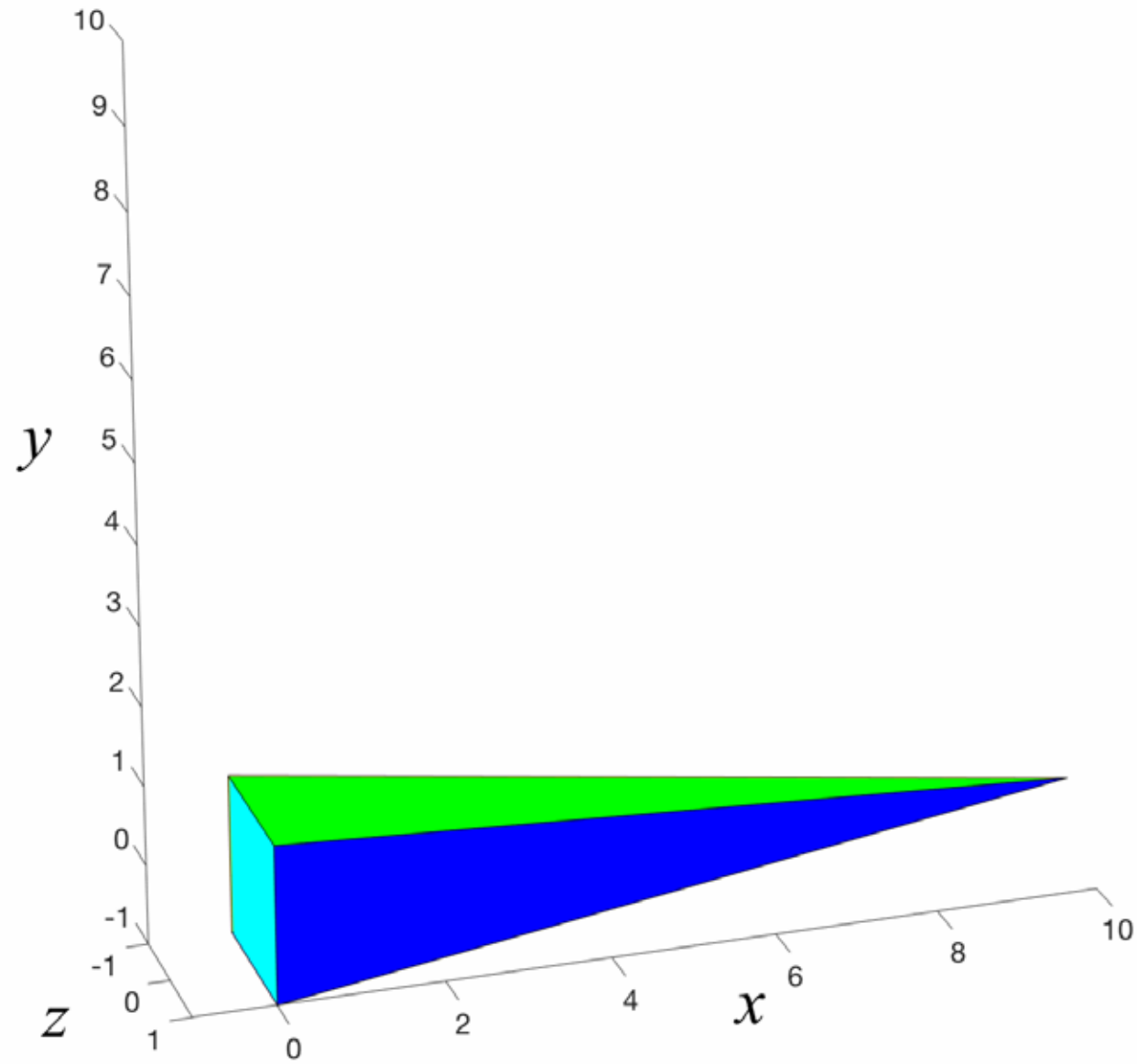
Bone of length $\ell = 10$



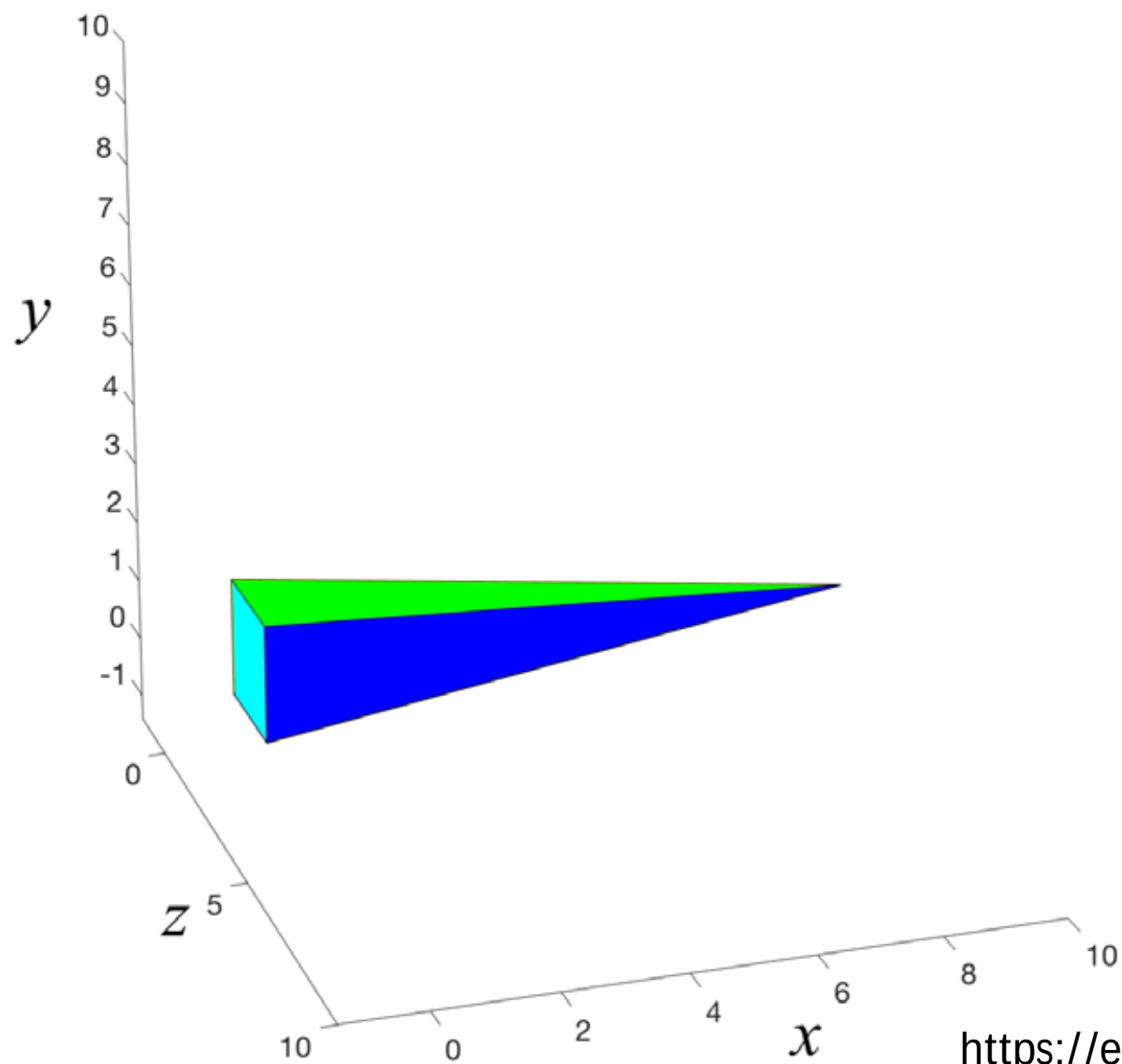
Twisting around x axis: $\theta_1 = 0^\circ$



Bending around z axis: $\theta_2 = 0^\circ$



Twist-bend-twist: $(\theta_1, \theta_2, \theta_3) = (0^\circ, 0^\circ, 0^\circ)$

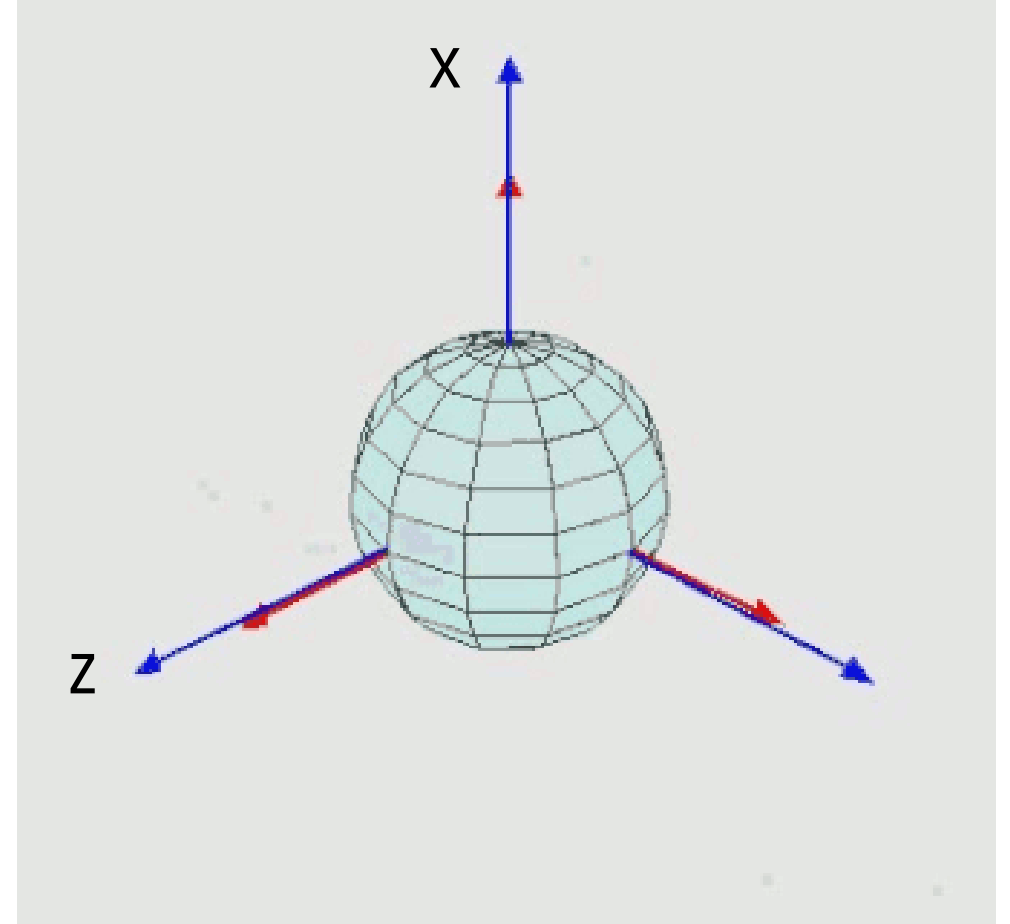


https://en.wikipedia.org/wiki/Euler_angles

<https://mathworld.wolfram.com/EulerAngles.html>

Euler Angle Rotations

$$R_x(q_3) * R_z(q_2) * R_x(q_1)$$



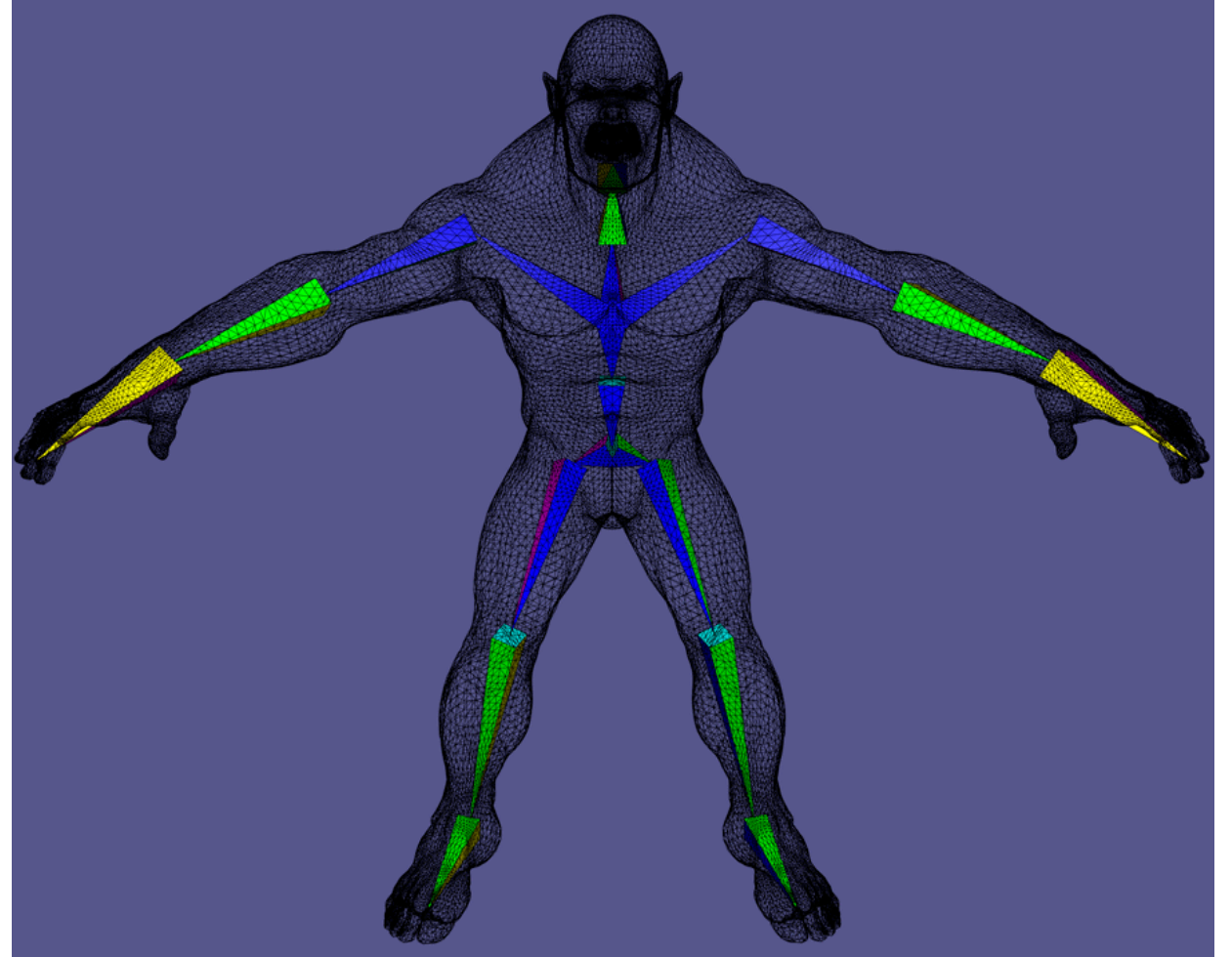
https://en.wikipedia.org/wiki/Euler_angles

<https://mathworld.wolfram.com/EulerAngles.html>

Skeletons: Forward Kinematics

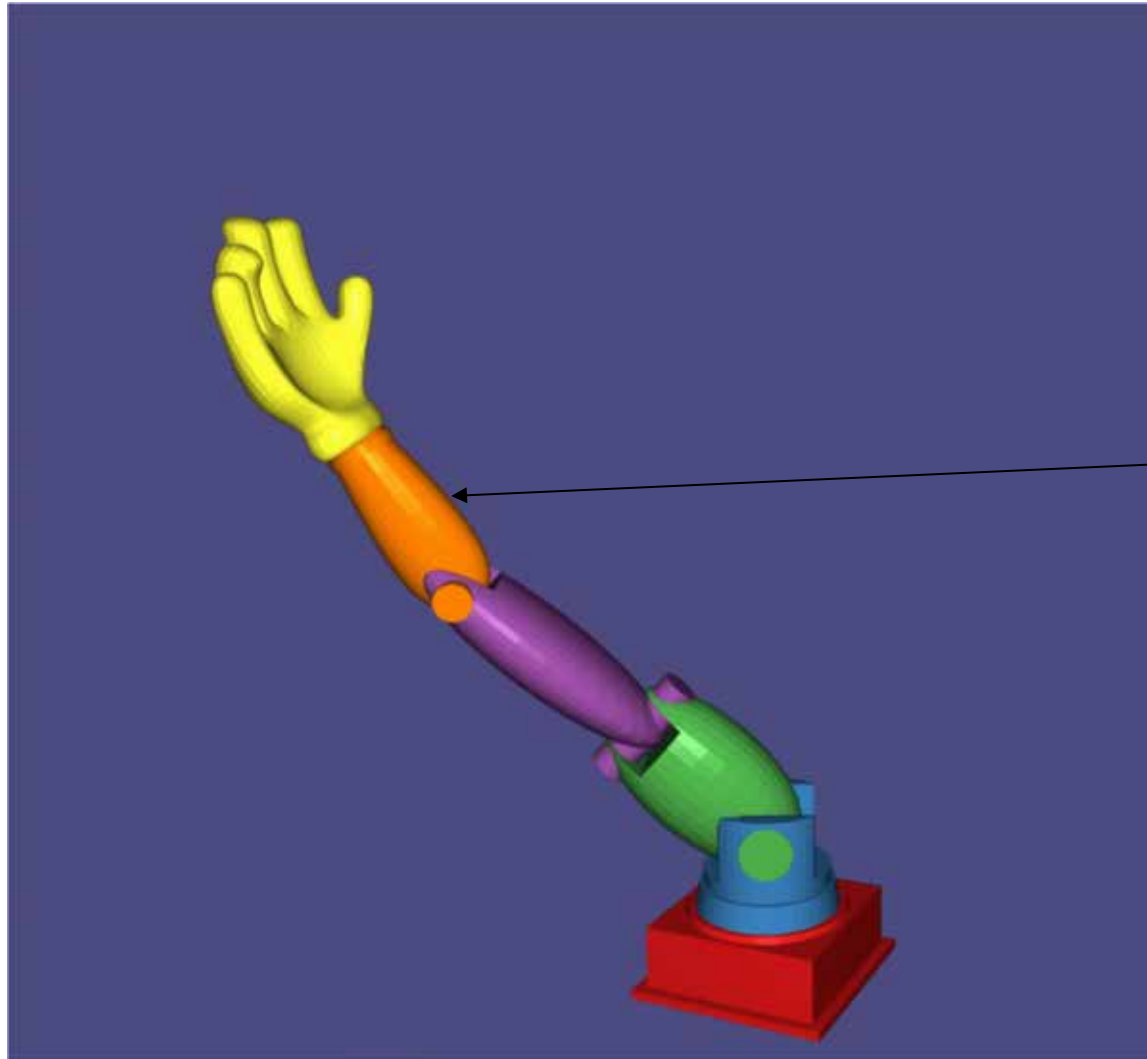
$$\mathbf{T}_i = \mathbf{T}_{p_i} \begin{pmatrix} \hat{\mathbf{T}}_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \overline{\mathbf{R}}_i & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \hat{\mathbf{T}}_i \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1}$$

$$\mathbf{T}_i = \mathbf{T}_{p_i} \hat{\mathbf{T}}_i \begin{pmatrix} \mathbf{R}_x(\theta_{i3}) & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_z(\theta_{i2}) & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_x(\theta_{i1}) & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \hat{\mathbf{T}}_i^{-1}$$



Rigid “Skinning”

Idea: Attach each vertex to a single bone



$$\mathbf{v}_j = \mathbf{T}_i \hat{\mathbf{v}}_j$$

Deformable Skinning

Rigid Skinning is fine for mechanical things, but for smoother deformations we need to try something else

Rather than attach each vertex to a single bone, we attach each vertex to multiple bones and ***blend*** their transformations

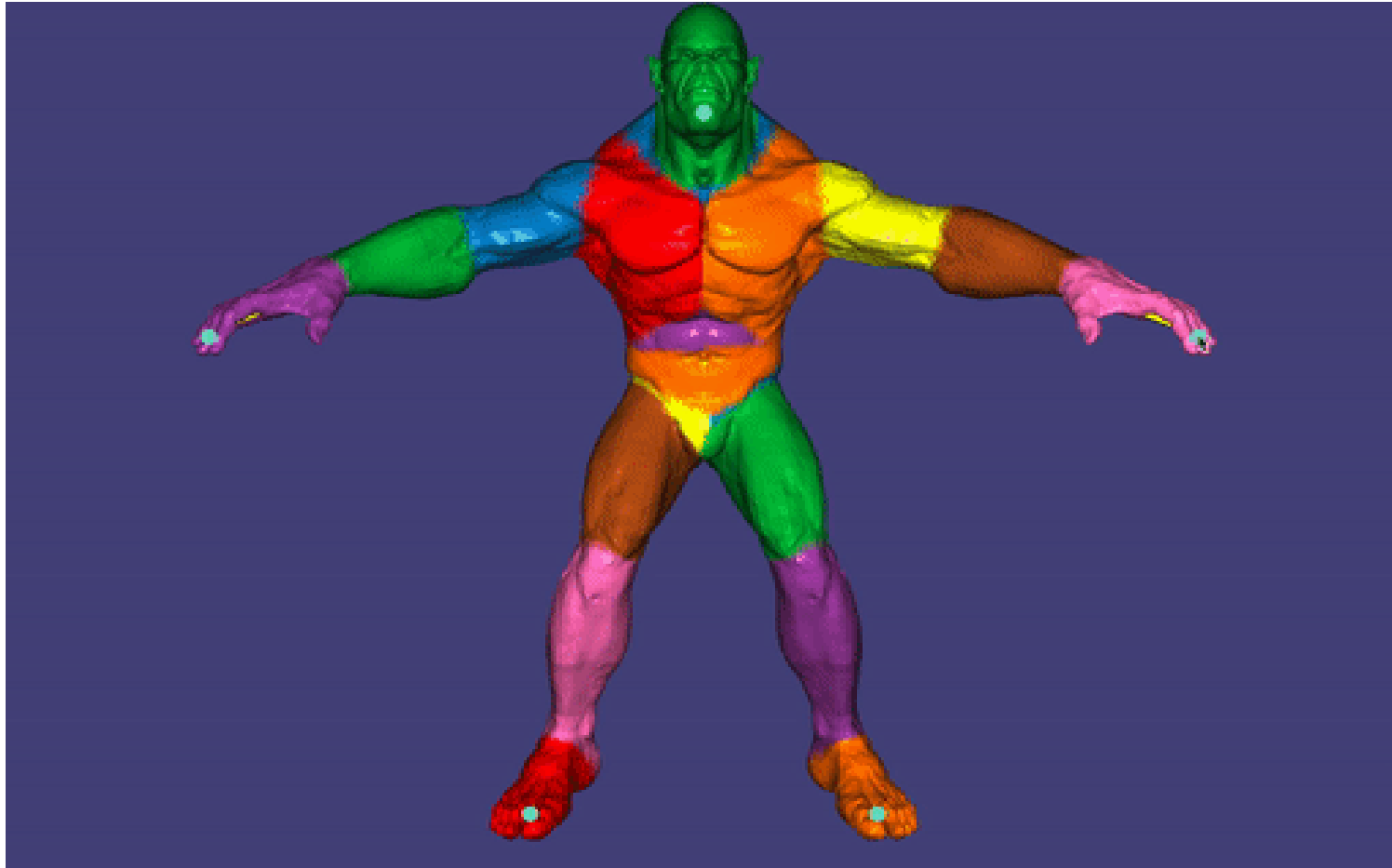
If this blending is linear in the transformations, we call it linear blend skinning.

Linear Blend Skinning

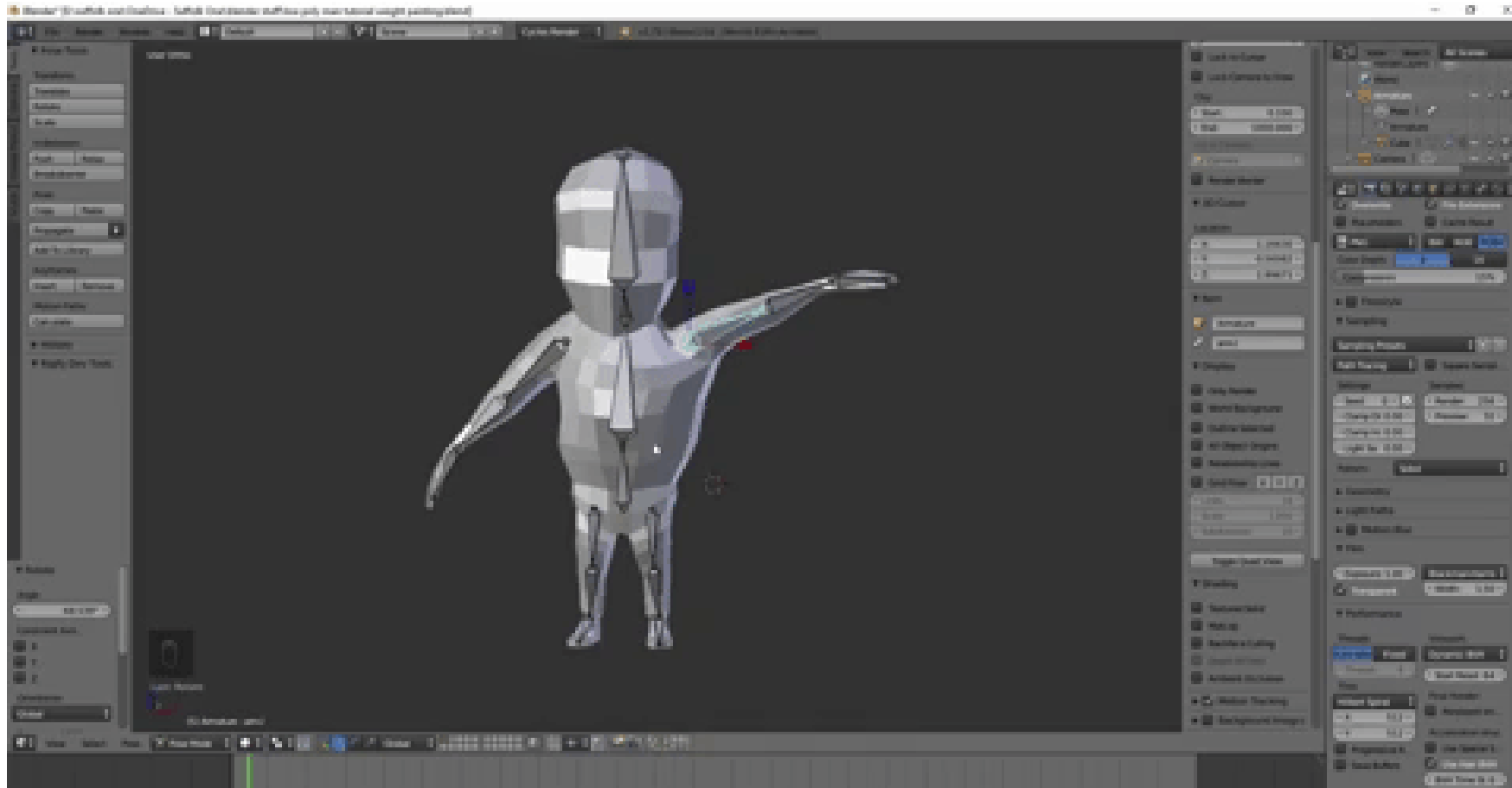


$$\mathbf{v}_j = \sum_{i=1}^{\text{\#bones}} w_{ij} \mathbf{T}_i \hat{\mathbf{v}}_j$$

Rigid vs Linear Blend Skinning

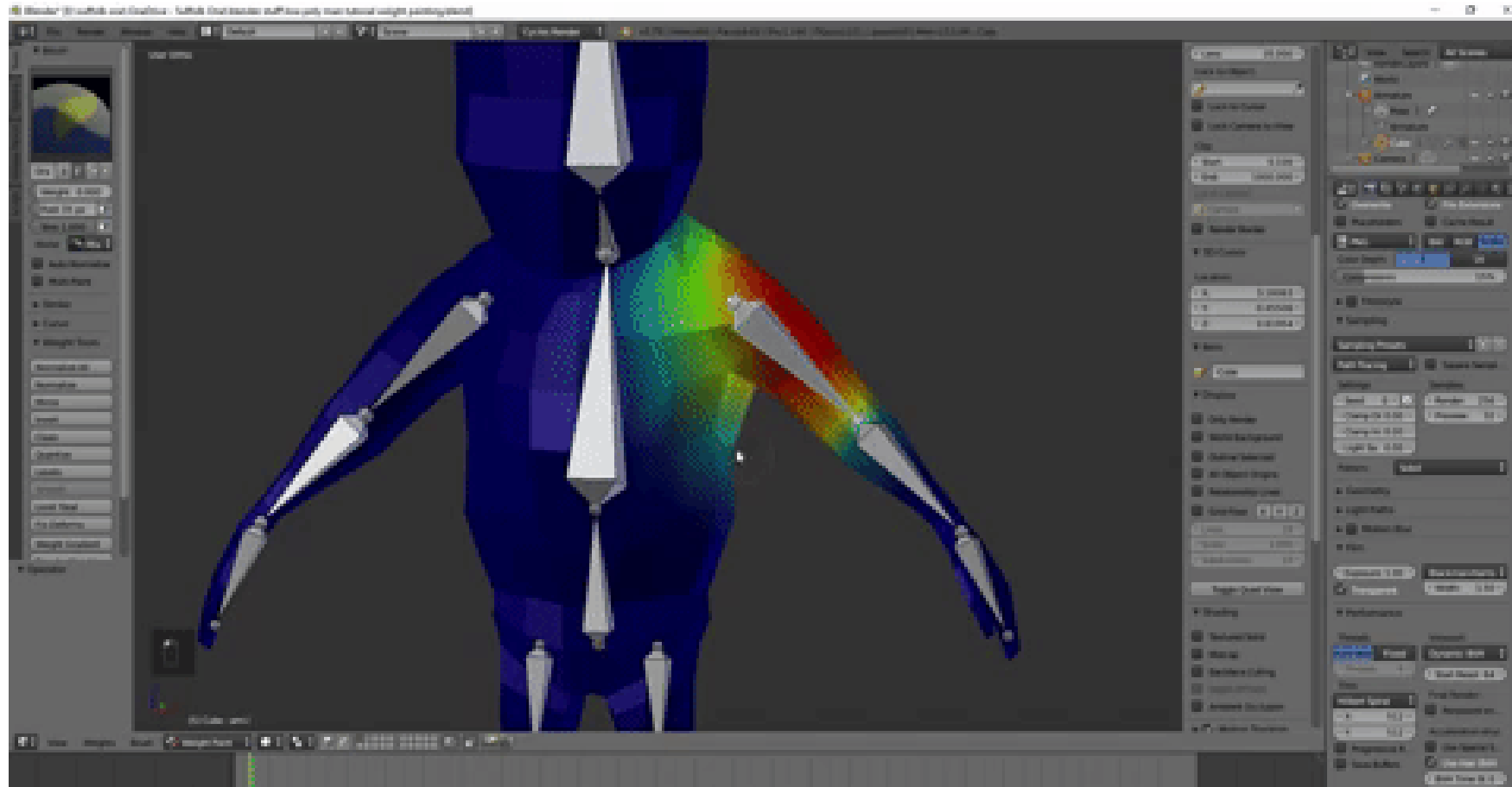


Weight Painting

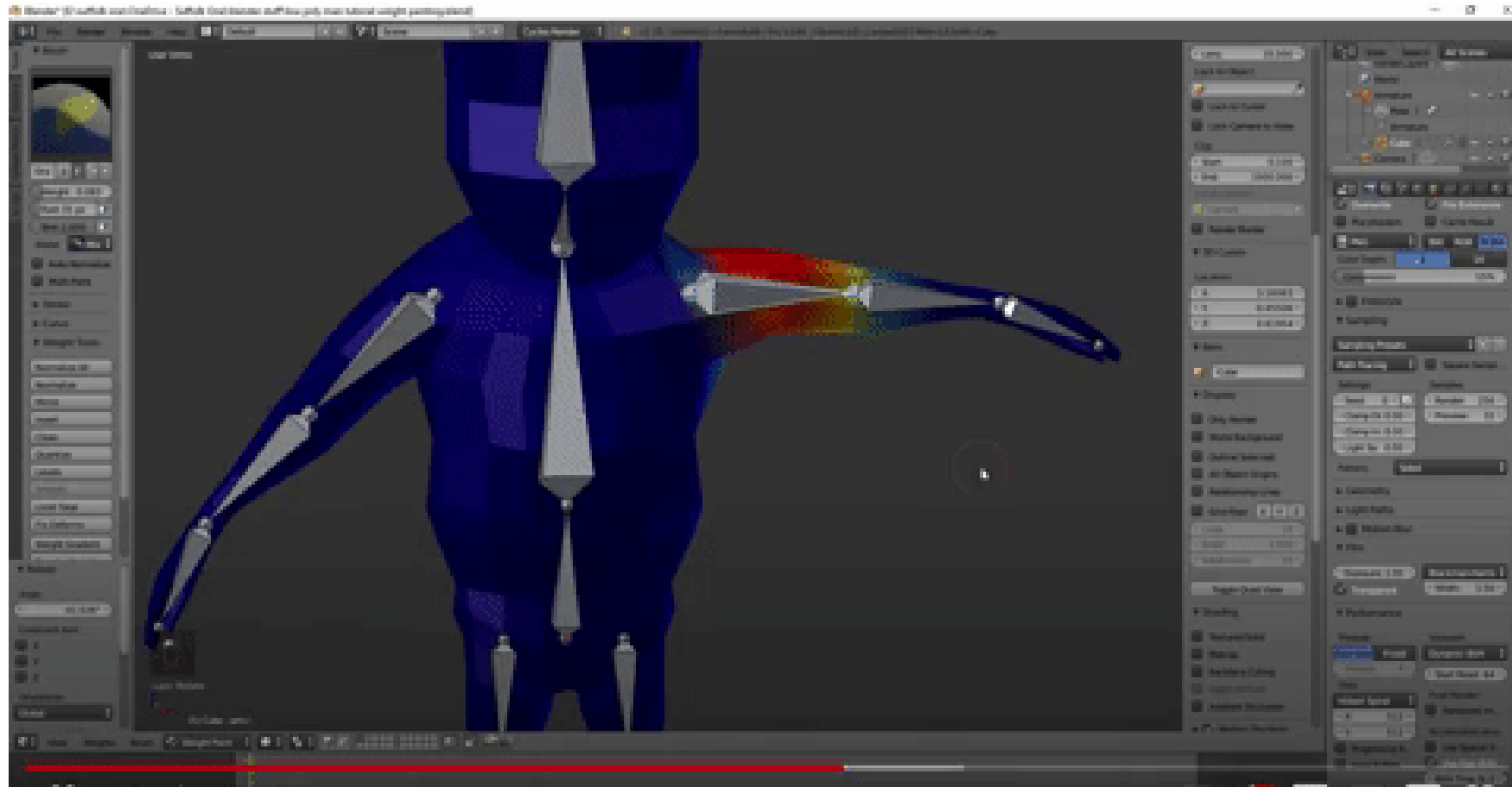


<https://www.youtube.com/watch?v=TI4qTgwQwYw>

Weight Painting



Weight Painting



<https://www.youtube.com/watch?v=TI4qTgwQwYw>

Specifying Keyframes



Time = 0



Time = 10s

Poses are generated by specifying rotations of bones

Each pose can be represented as

$$\theta = \begin{pmatrix} \theta_{11} \\ \theta_{11} \\ \theta_{11} \\ \vdots \\ \theta_{n1} \\ \theta_{n2} \\ \theta_{n3} \end{pmatrix}$$

Specifying Keyframes



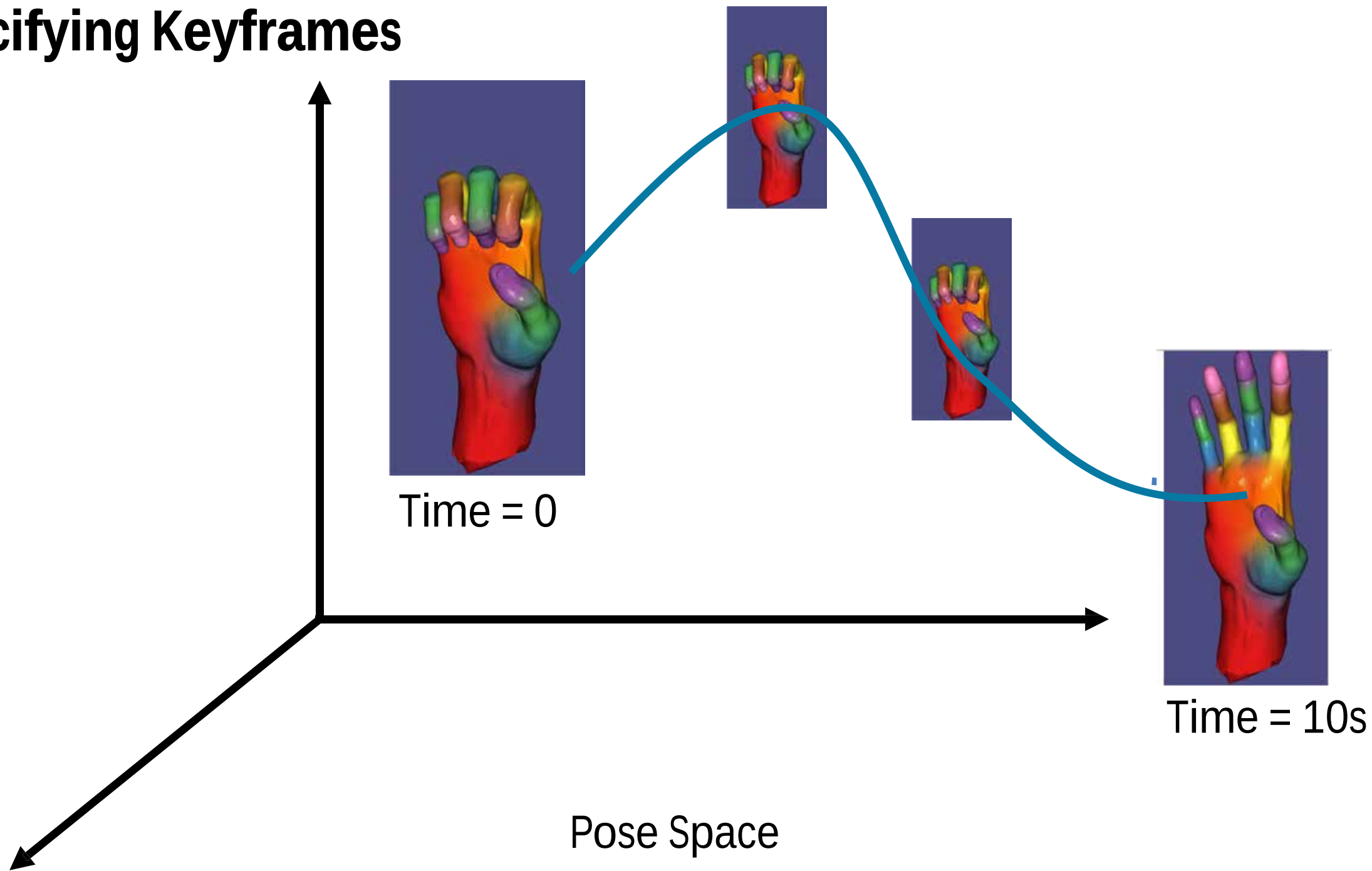
Time = 0

????????????



Time = 10s

Specifying Keyframes



Specifying Keyframes



https://en.wikipedia.org/wiki/Twelve_basic_principles_of_animation#Slow_in_and_slow_out

<https://www.youtube.com/watch?v=fQBFsTqbKhY>

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

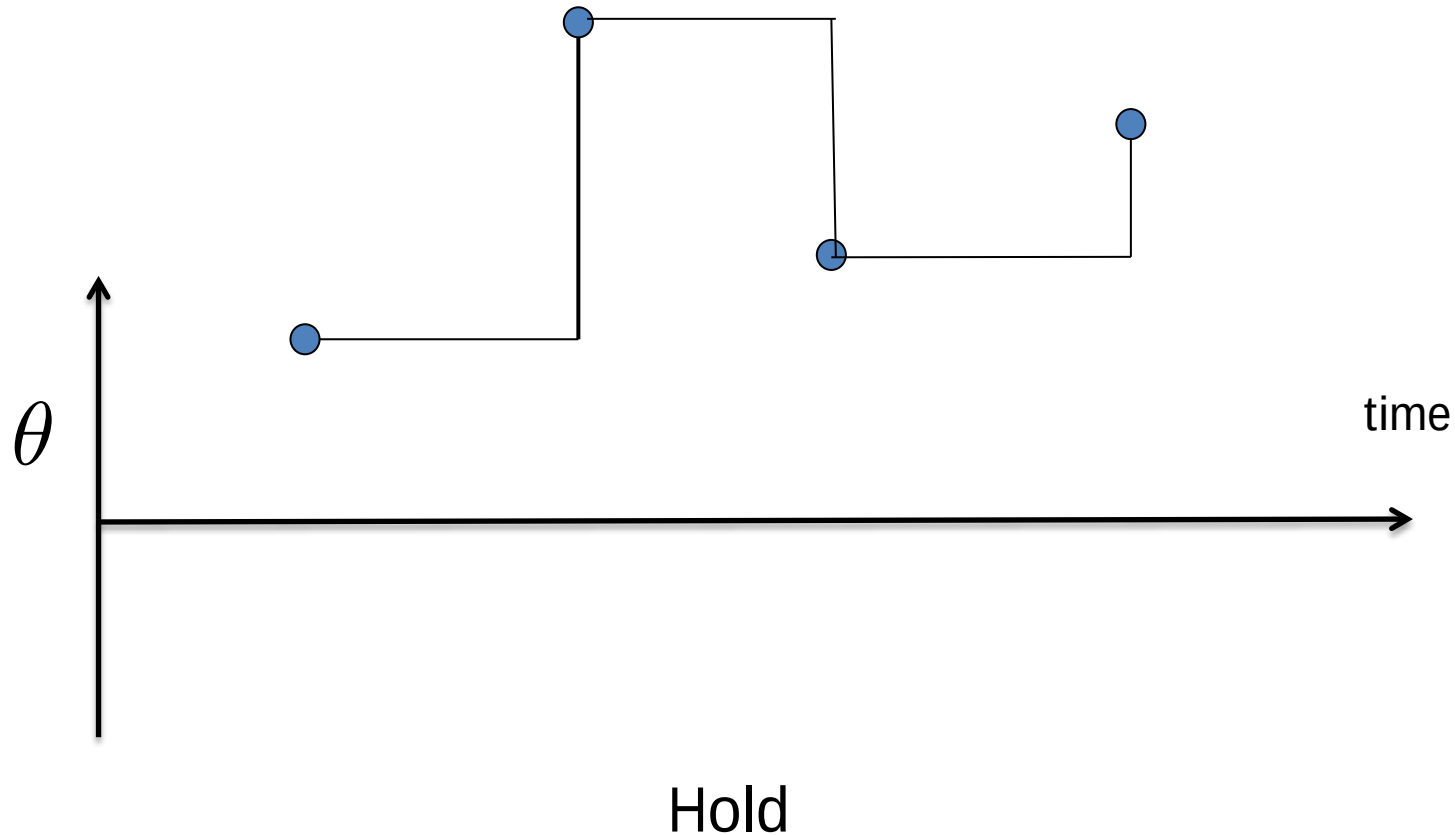
How could we construct such a curve from keyframes ?

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?

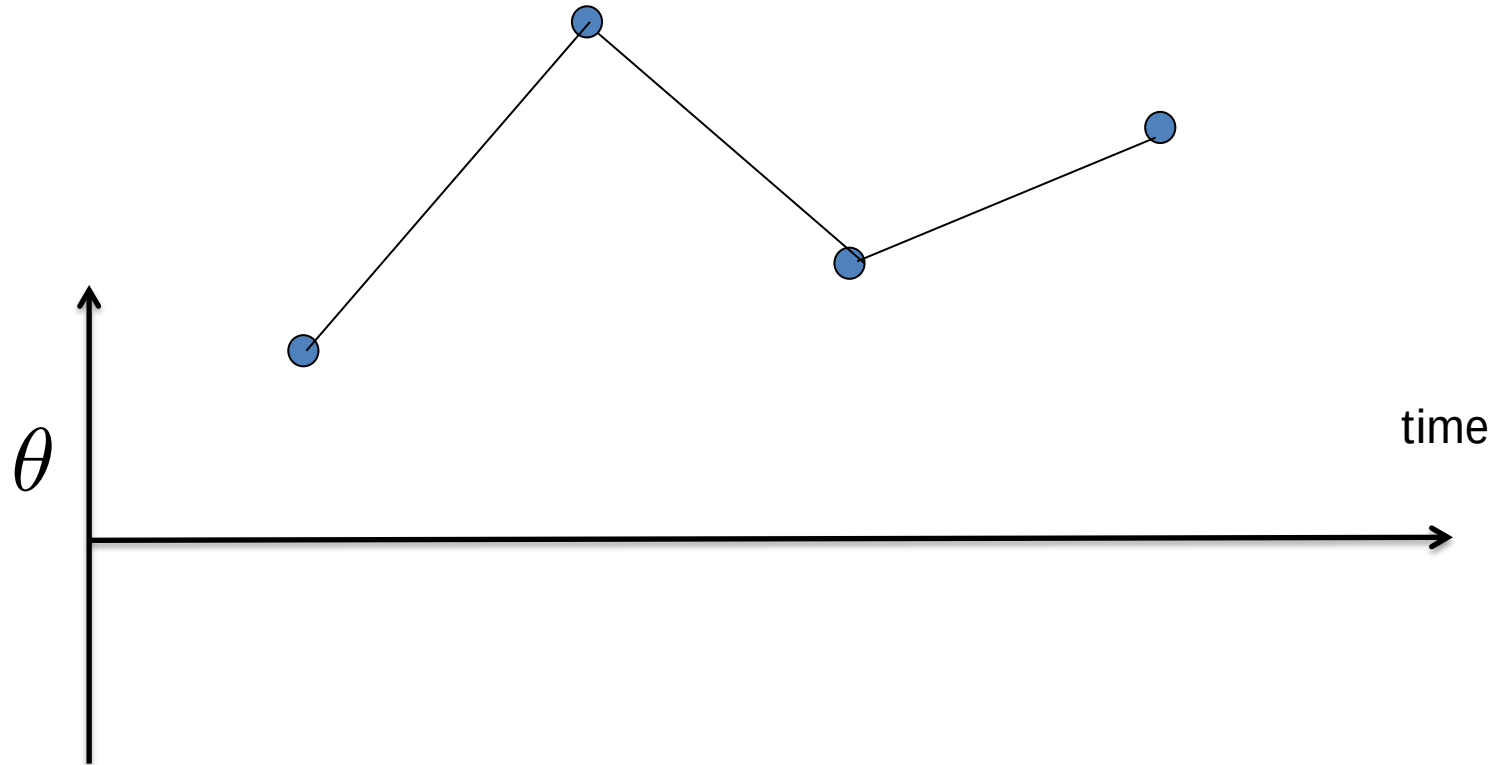


Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?



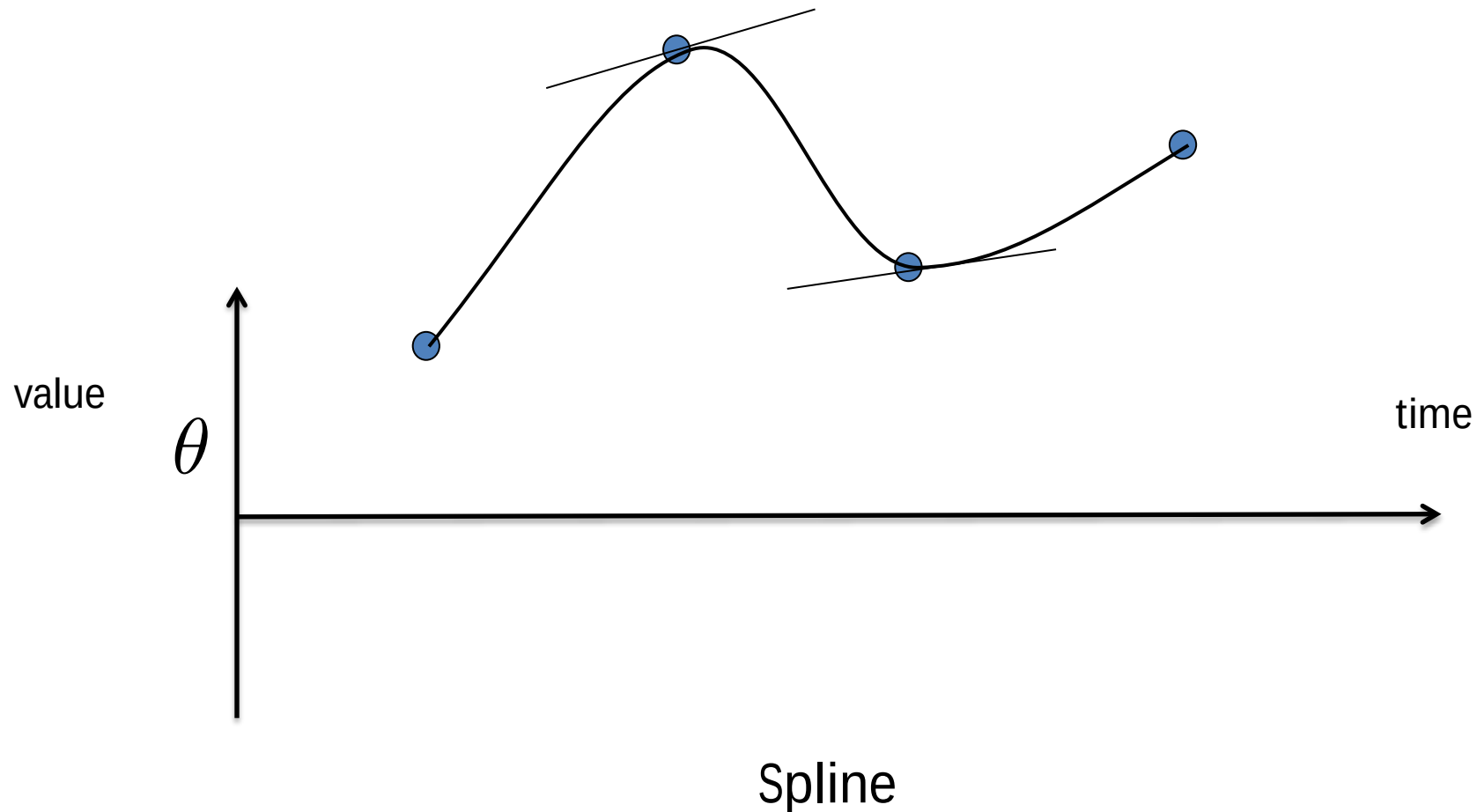
Linear

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?



Catmull-Rom Spline

A **cubic** curve created by specifying the end points and the tangents of the curve.

$$\mathbf{c}(t) = at^3 + bt^2 + ct + d$$
$$\mathbf{c}'(t) = 3at^2 + 2bt + c$$



Catmull-Rom Spline

A **cubic** curve created by specifying the end points and the tangents of the curve.

$$c(0) = d$$

$$c(1) = a + b + c + d$$

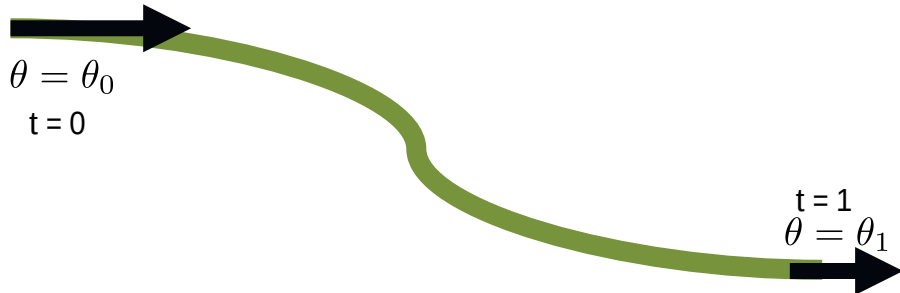
$$\frac{dc}{dt}(0) = 1c$$

$$\frac{dc}{dt}(1) = 3a + 2b + 1c$$

$$c(t) = [t^3 \ t^2 \ t \ 1] A$$

$$c'(t) = [3t^2 \ 2t \ 1 \ 0] A$$

$$\text{where } A = \begin{pmatrix} \mathbf{a}^T \\ \mathbf{b}^T \\ \mathbf{c}^T \\ \mathbf{d}^T \end{pmatrix}$$



Catmull-Rom Spline

$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix} \begin{pmatrix} \mathbf{a}^T \\ \mathbf{b}^T \\ \mathbf{c}^T \\ \mathbf{d}^T \end{pmatrix} = \begin{pmatrix} \theta_0^T \\ \theta_1^T \\ \mathbf{m}_0^T \\ \mathbf{m}_1^T \end{pmatrix}$$

$$\mathbf{c}(t) = [t^3 \ t^2 \ t \ 1] \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix}^{-1} \begin{pmatrix} \theta_0^T \\ \theta_1^T \\ \mathbf{m}_0^T \\ \mathbf{m}_1^T \end{pmatrix}$$

Catmull-Rom Spline

After solving and rearranging we end up with

$$\mathbf{c}(t) = (2t^3 - 3t^2 + 1)\theta_0 + (t^3 - 2t^2 + t)\mathbf{m}_0 + (-2t^3 + 3t^2)\theta_1 + (t^3 - t^2)\mathbf{m}_1$$

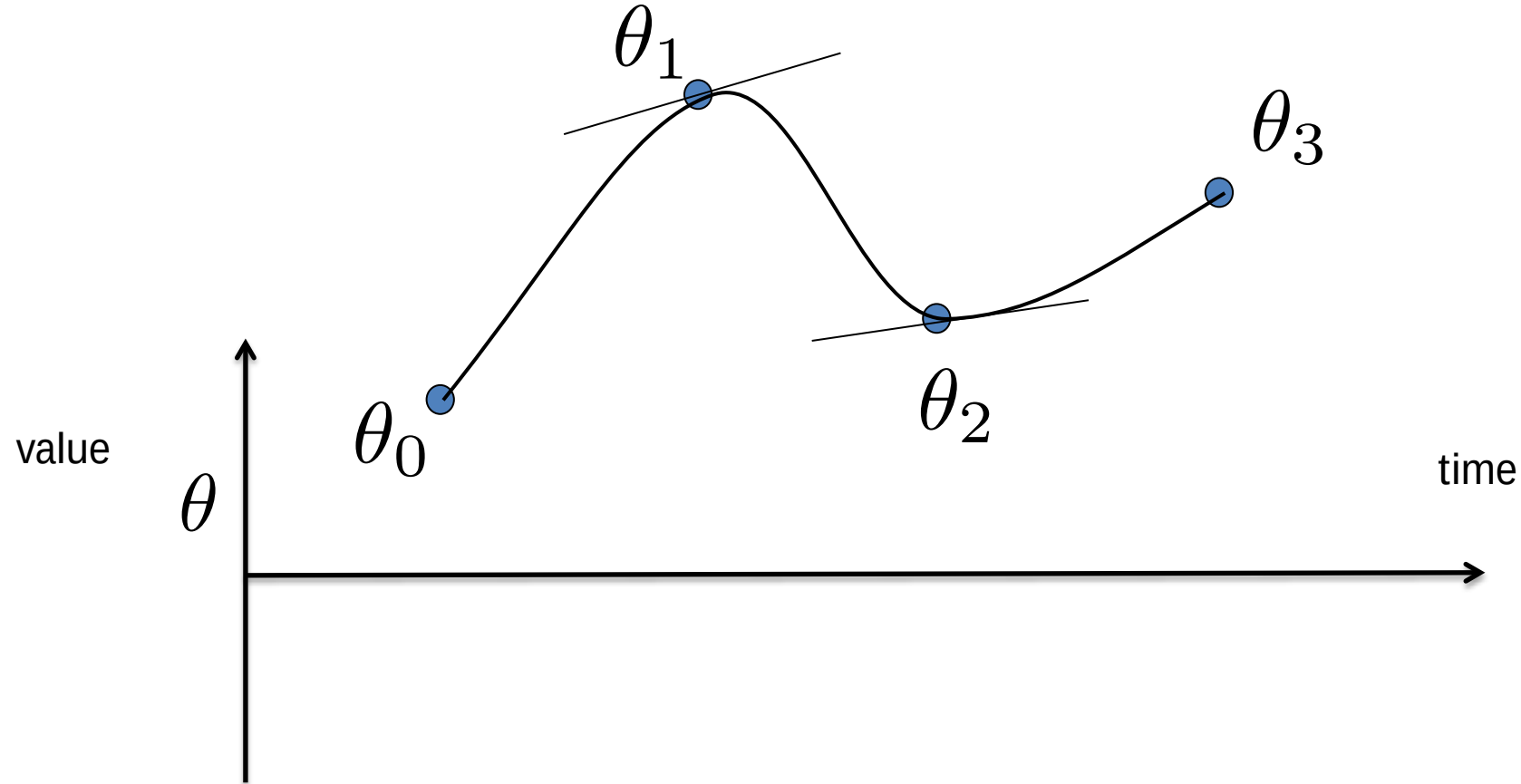
Remember this is for $t = 0$ to $t = 1$.

For arbitrary intervals substitute

$$t = (t' - t_0)/(t_1 - t_0)$$

Catmull-Rom Spline

Catmull-Rom chooses the tangents using “Finite Differences”



$$\mathbf{m}_k = \frac{\theta_{k+1} - \theta_k}{t_{k+1} - t_k}$$

Catmull-Rom Animation

For each time, t , create a new $\theta(t)$ using your spline.

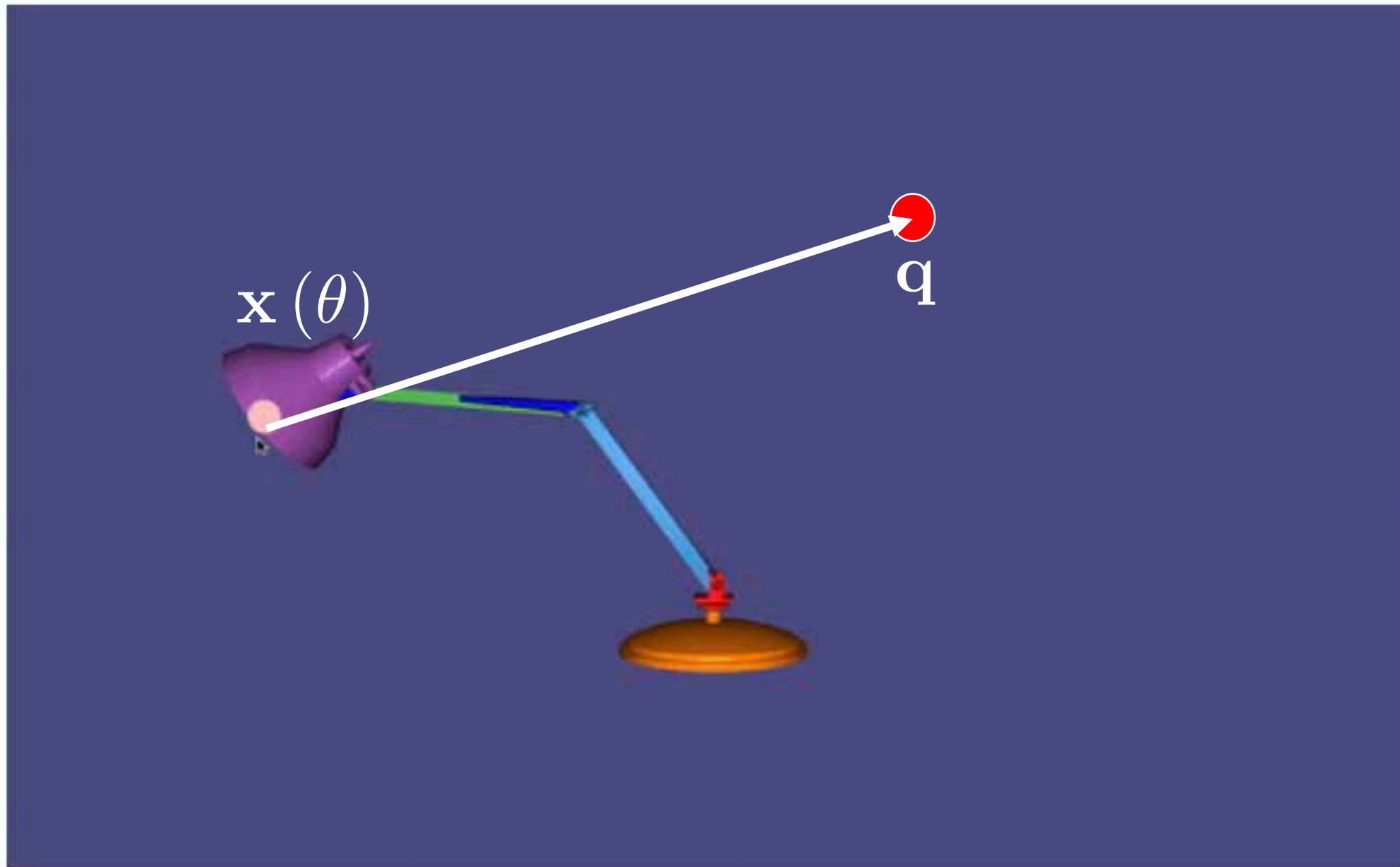
Re-pose your bones using θ , deform skin and draw...



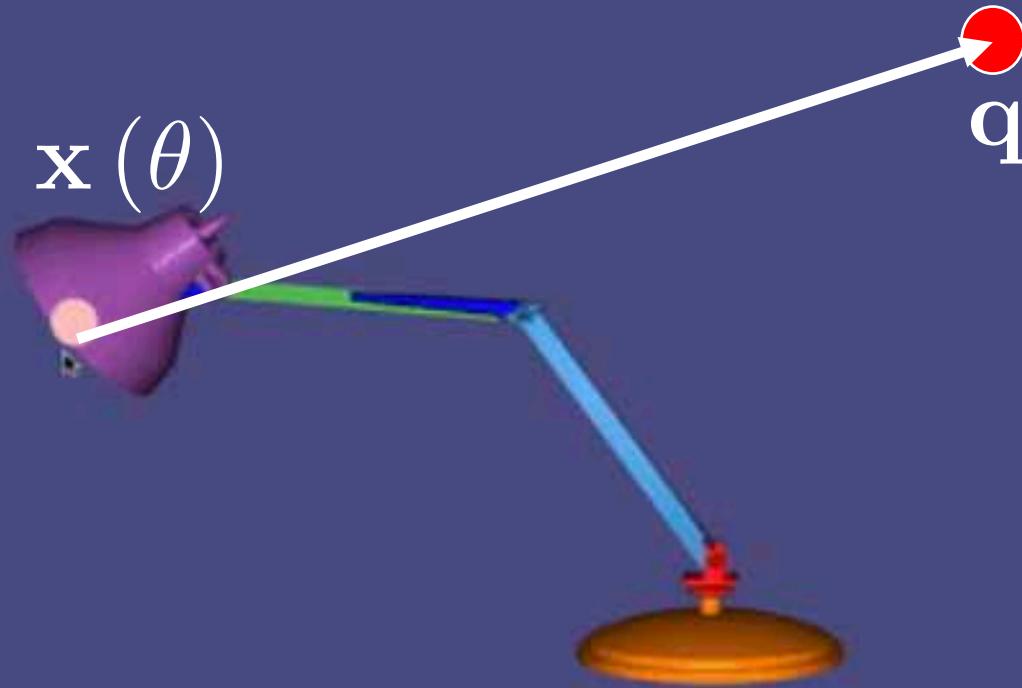
Inverse Kinematics



Inverse Kinematics



Inverse Kinematics



$$\theta^* = \arg \min \|\mathbf{x}(\theta) - \mathbf{q}\|_2^2$$

Skeletons: Inverse Kinematics

What is the pose (set of joint angles $\mathbf{a}$) that lets us reach a given point (end-effector position).

$$\mathbf{a} = \begin{pmatrix} \theta_{11} \\ \theta_{12} \\ \theta_{13} \\ \theta_{21} \\ \theta_{22} \\ \theta_{23} \\ \vdots \\ \theta_{m1} \\ \theta_{m2} \\ \theta_{m3} \end{pmatrix}$$



What does it mean to reach (get as close as possible) to a point?

Skeletons: Inverse Kinematics

Closeness energy can be measured the squared distance between the pose tip x_b of some bone b and a desired goal location $q \in \mathbb{R}^3$.

$$E(\mathbf{x}_b) = \|\mathbf{x}_b - \mathbf{q}\|^2.$$

Given pose vector $\mathbf{a}$, the bone tip x_b is:

$$\mathbf{x}_b(\mathbf{a}) = \mathbf{T}_b \hat{\mathbf{d}}_b \quad \text{Reminder: Forward Kinematics}$$

Now given any number of end-effectors $b_1, \dots, b_k$:

$$\min_{\mathbf{a}} \underbrace{\sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2}_{E(\mathbf{x}_b(\mathbf{a}))}$$

And we impose some joint angle limits: $\min_{\mathbf{a}^{\min} \leq \mathbf{a} \leq \mathbf{a}^{\max}} E(\mathbf{x}_b(\mathbf{a}))$

Minimizing this energy is a non-linear (x is non-linear in a) least squares (sum of squares of x_b) problem.



Inverse Kinematics: Energy

$$E(\mathbf{x}_b(\mathbf{a})) = \|\mathbf{x}_b(\mathbf{a}) - \mathbf{q}\|^2$$

list of constrained end
effectors

→ $b = \{b_1, b_2, \dots, b_k\}$

$$\min_{\mathbf{a}} \underbrace{\sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2}_{E(\mathbf{x}_b(\mathbf{a}))}$$

Inverse Kinematics: Energy

Over all choices of Euler angles $\mathbf{a}$, we want the angles that ensure all selected end effectors go to their prescribed locations, subject to angle constraints:

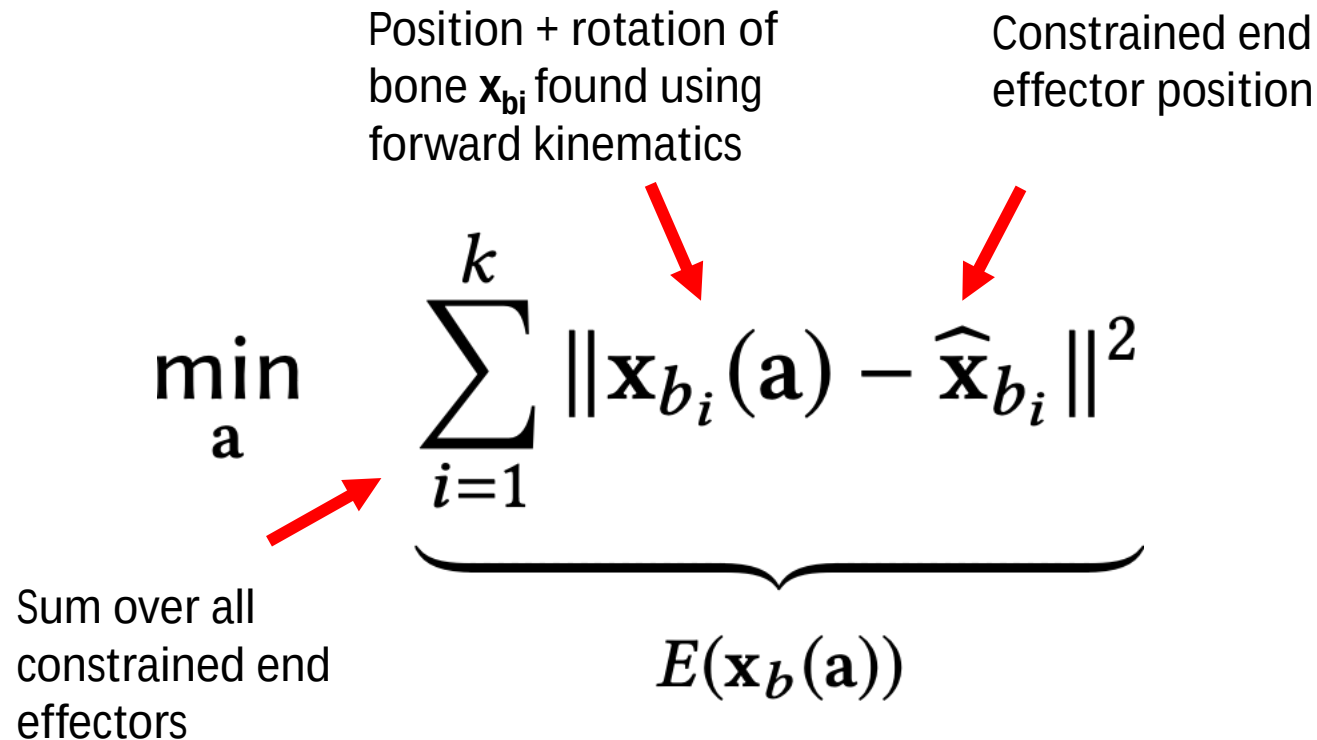
$$\min_{\mathbf{a}^{\min} \leq \mathbf{a} \leq \mathbf{a}^{\max}} E(\mathbf{x}_b(\mathbf{a}))$$

Position + rotation of bone $\mathbf{x}_{b_i}$ found using forward kinematics

Constrained end effector position

$\min_{\mathbf{a}}$

Sum over all constrained end effectors

$$\underbrace{\sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2}_{E(\mathbf{x}_b(\mathbf{a}))}$$


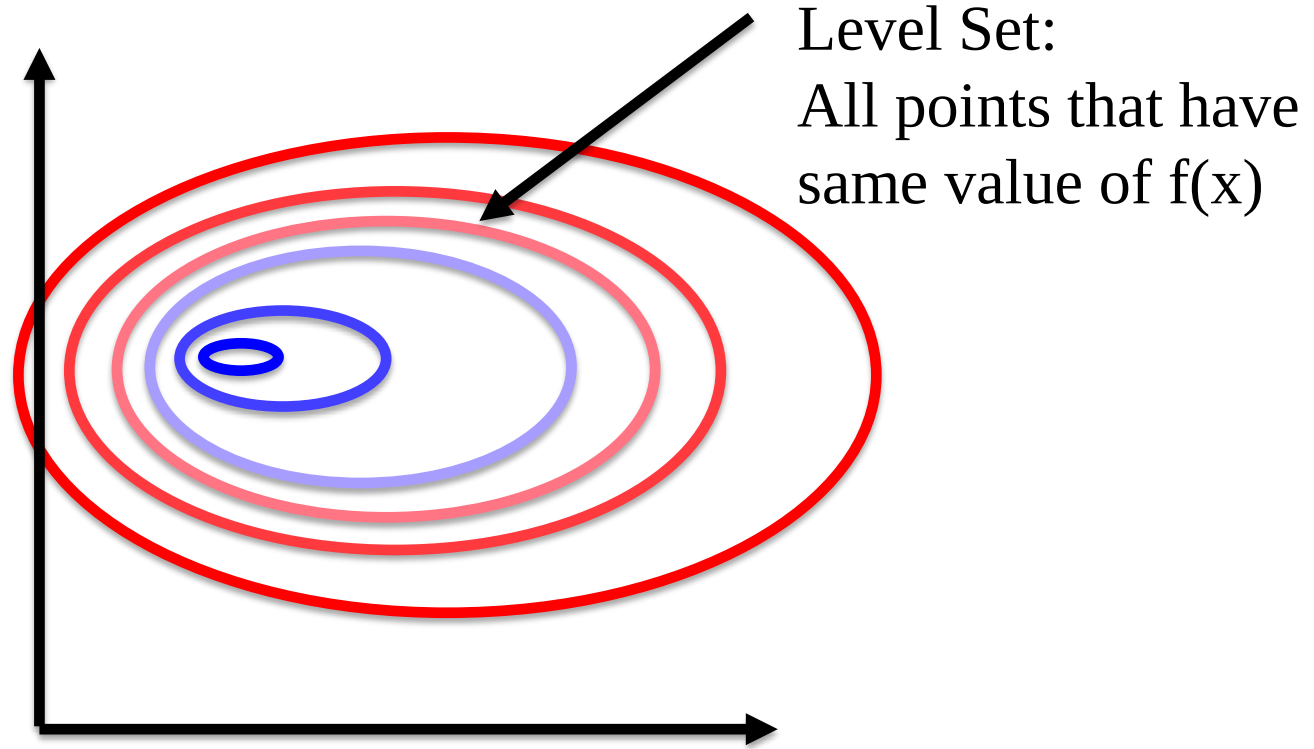
Gradient Descent

- Make a guess.
- Iteratively move in a direction lower energy.

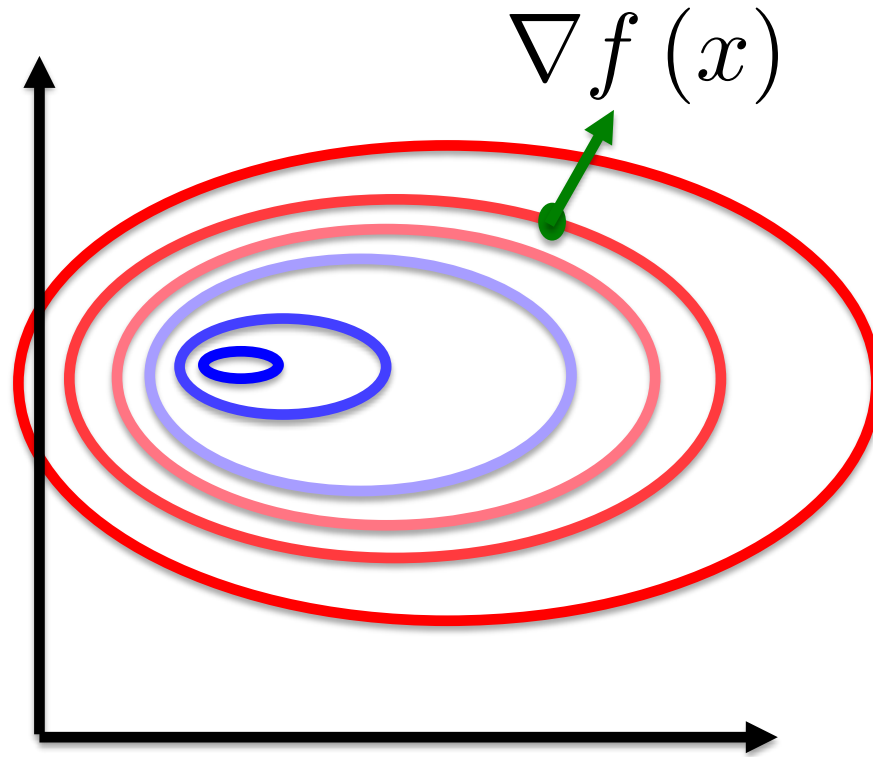
Recall that the gradient of a function points in direction of maximum ascent

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial \mathbf{x}_1} \quad \frac{\partial f}{\partial \mathbf{x}_2} \quad \cdots \quad \frac{\partial f}{\partial \mathbf{x}_n} \right)$$

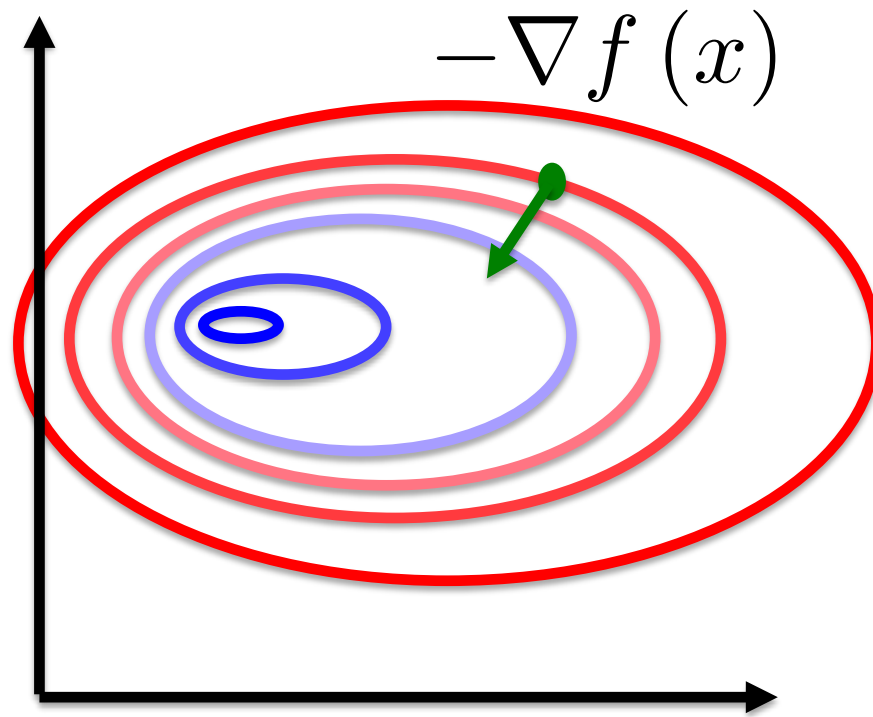
An Aside: Level Sets



Gradient Descent



Gradient Descent

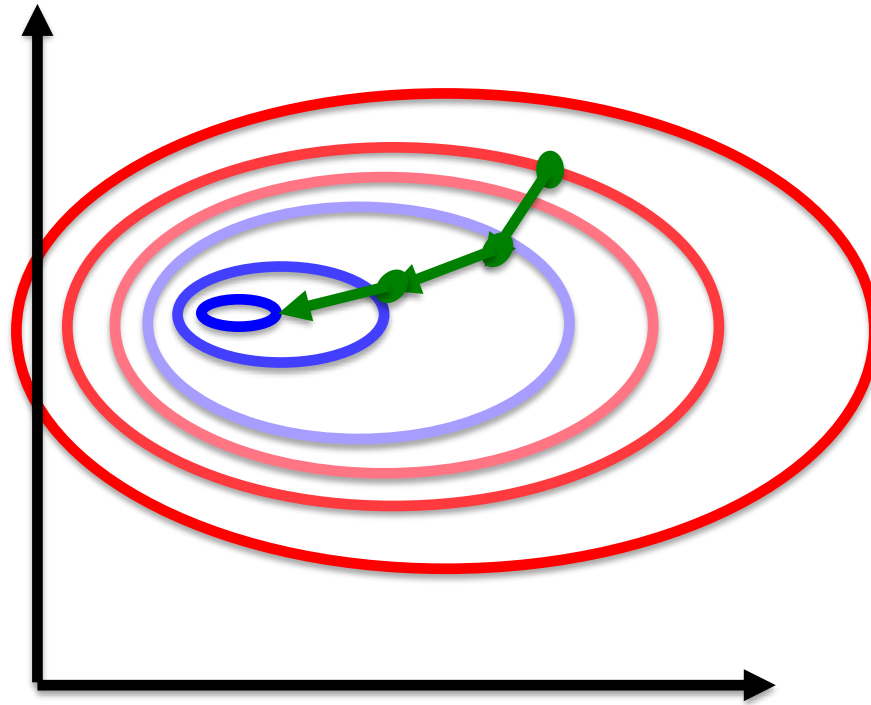


Projected Gradient Descent Algorithm

While not at an optimal point

- Compute the gradient at current point (x)
- Move to new point $x = x - h \nabla f(x)$
- Project x to satisfy any constraints like joint angle limits.
- Iterate to a minima.

Gradient Descent



Gradient Descent

So let's take a step in the negative gradient direction of the objective

Goal

$$\min_{\mathbf{a}} \underbrace{\sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \widehat{\mathbf{x}}_{b_i}\|^2}_{E(\mathbf{x}_b(\mathbf{a}))}$$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{dE(\mathbf{x}(\mathbf{a}))}{d\mathbf{a}} \right)^T$$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{d\mathbf{x}(\mathbf{a})}{d\mathbf{a}} \right)^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right)$$

Gradient Descent

So let's take a step in the negative gradient direction of the objective

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{dE(\mathbf{x}(\mathbf{a}))}{d\mathbf{a}} \right)^T$$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{d\mathbf{x}(\mathbf{a})}{d\mathbf{a}} \right)^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right)$$

Gradient Descent: Kinematic Jacobian

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{d\mathbf{x}(\mathbf{a})}{d\mathbf{a}} \right)^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right)$$

The change in tip positions $\mathbf{x}$ with respect to joint angles $\mathbf{a}$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \mathbf{J}^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right)$$

Computed using finite differences

Line Search

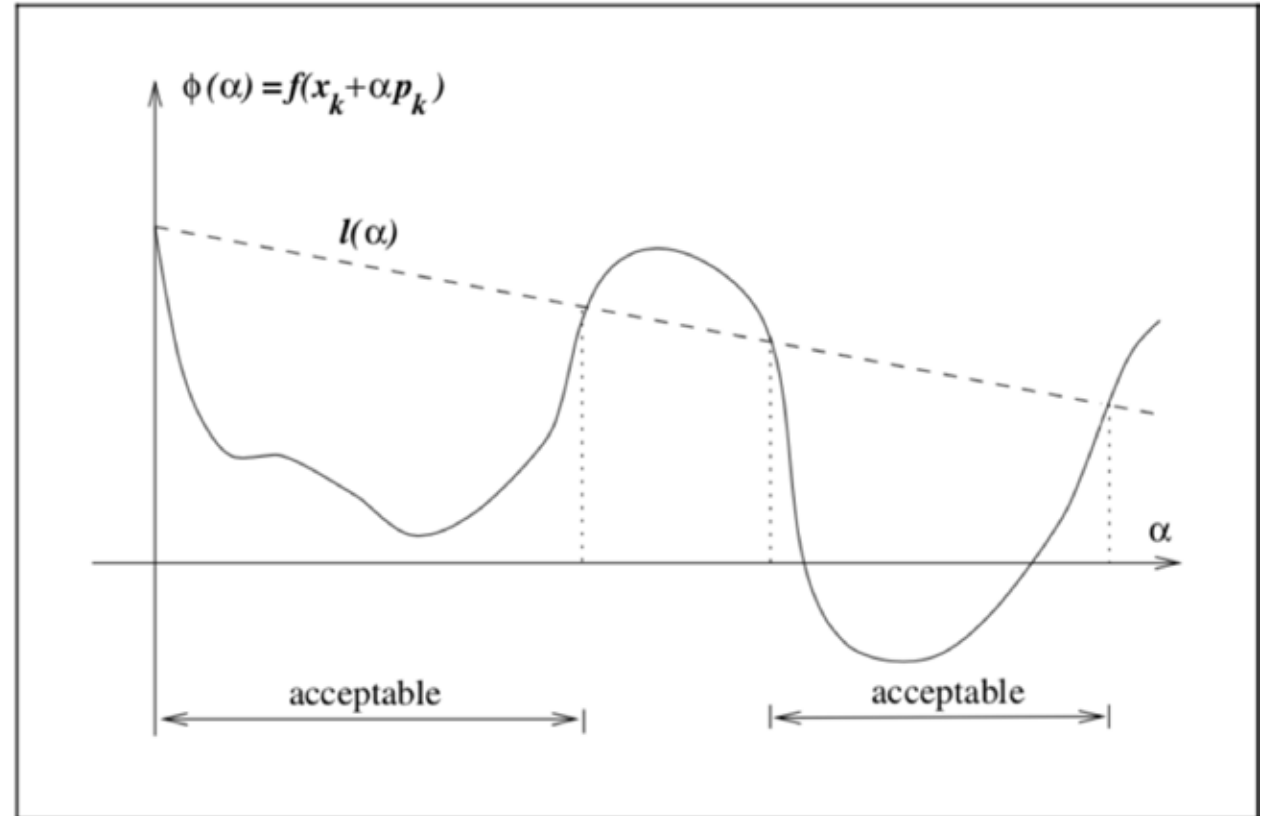
$$\mathbf{a} \leftarrow \mathbf{a} - \boxed{\sigma} \mathbf{J}^\top \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right)$$

AKA we are moving in descent direction and then projecting:

$$\mathbf{a} \leftarrow \text{proj}(\mathbf{a} + \Delta \mathbf{a})$$

Start with large σ
and decrease by $\frac{1}{2}$ until

$$E(\text{proj}(\mathbf{a} + \sigma \Delta \mathbf{a})) < E(\mathbf{a})$$



Skeletons: Inverse Kinematics Minimization

Projected Gradient Descent:

Start with an initial pose $\mathbf{a}$, and move in direction of decrease in E , project the pose to stay within limits and iterate towards solution.

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{dE(\mathbf{x}(\mathbf{a}))}{d\mathbf{a}} \right)^T$$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{d\mathbf{x}(\mathbf{a})}{d\mathbf{a}} \right)^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right) \quad \text{chain rule}$$

$$\frac{dE}{d\mathbf{a}} \in \mathbb{R}^{|\mathbf{a}|}, \quad \frac{dE}{d\mathbf{x}} \in \mathbb{R}^{|\mathbf{x}|}, \quad \text{and} \quad \frac{d\mathbf{x}}{d\mathbf{a}} \in \mathbb{R}^{|\mathbf{x}| \times |\mathbf{a}|}$$

$$\mathbf{J} = \frac{d\mathbf{x}}{d\mathbf{a}}. \quad \text{also known as Jacobian measures the change in } \mathbf{x} \text{ for changes in joint angles } \mathbf{a},$$

$$\mathbf{J} \text{ can be computed using Finite Differences:} \quad \mathbf{J}_{i,j} \approx \frac{\mathbf{x}_i(\mathbf{a} + h\delta_j)}{h}. \quad h = 10^{-7}$$

$$\left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right) \text{ is gradient of } \sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2$$

$$\text{Project to within limits: } \mathbf{a}_i \leftarrow \max[\mathbf{a}_i^{\min}, \min[\mathbf{a}_i^{\max}, \mathbf{a}_i]].$$

$$\text{Find a good step that lowers energy: } E(\text{proj}(\mathbf{a} + \sigma \Delta \mathbf{a})) < E(\mathbf{a}).$$

Assignment #7 Tasks

File	Marks	What it does
src/euler_angles_to_transform.py	17	XZX Euler angles $\rightarrow$ 4x4 rotation transform.
src/forward_kinematics.py	17	Global pose transform of every bone (recursive).
src/linear_blend_skinning.py	17	Weighted blend of bone transforms to deform a mesh.
src/copy_skeleton_at.py	17	Deep-copy a skeleton with new joint angles from a flat vector.
src/transformed_tips.py	17	End-effector tip positions after forward kinematics.
src/kinematics_jacobian.py	5	Finite-difference kinematic Jacobian $\partial \mathbf{x}_b / \partial \theta$.
src/line_search.py	4	Backtracking line search for a decreasing step.
src/end_effectors_objective_and_gradient.py	3	IK objective f , gradient grad_f , and projection proj_z .
src/projected_gradient_descent.py	3	Projected gradient descent loop.

Iterate over bones i , compute $T_i = T_p * T_{\text{local } i} * T_r^{-1}$
call p recursively from i if not yet computed.
compute FK and then iteratively transform tips b_i , to
compute x_{b_i}

Next: Simulation, mass-spring systems

